ĐẠI HỌC QUỐC GIA HÀ NỘI
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ
KHOA CÔNG NGHỆ THÔNG TIN

BÁO CÁO BÀI TẬP LỚN

Môn học: Lập trình nhúng (INT3108)

Hệ thống bãi đỗ xe thông minh sử dụng ESP32-C3 và ESP8266

Giảng viên hướng dẫn: Trần Hoàng Việt
Lớp môn học: INT3108 1

Nhóm sinh viên thực hiện

Họ và tên	MSSV
Nguyễn Chí Minh	24020232
Đinh Đức Thịnh	24020313
Nguyễn Việt Tiến	24020322

Hà Nội, tháng 10 năm 2026

Tóm tắt

Báo cáo trình bày việc phân tích, thiết kế, hiện thực và kiểm thử một **hệ thống bãi đỗ xe thông minh** quy mô mô hình gồm 5 chỗ đỗ và 2 cổng ra/vào. Hệ thống gồm ba vi điều khiển Wi-Fi của Espressif phối hợp qua một mạng Wi-Fi cục bộ:

- **Máy chủ** ESP32-C3 SuperMini chạy ESP-IDF 6.1 và hệ điều hành thời gian thực FreeRTOS: nhận dữ liệu qua UDP, quản lý phiên gửi xe và tính phí, điều khiển hai thanh chắn servo và còi báo cháy, hiển thị trên màn hình TFT ST7789 và phục vụ trang web giám sát tại `http://parking.local` (mDNS).
- **Sensor Client** (ESP8266) đọc 5 cảm biến hồng ngoại, cảm biến lửa và cảm biến mực nước; gửi trạng thái ngay khi có thay đổi và nhắc lại mỗi giây.
- **RFID Client** (ESP8266) phát điểm truy cập Wi-Fi, đọc hai đầu đọc RFID MFRC522 (cổng vào, cổng ra) và cảm biến nhiệt độ DHT11.

Báo cáo đi theo đúng yêu cầu của môn học: giới thiệu bài toán, 10 use case chi tiết, các thành phần hệ thống, hướng dẫn cài đặt môi trường, thiết kế và phân tích kiến trúc (kèm demo), hướng dẫn chạy mã nguồn từng bước, kiểm thử, kết luận và phần đóng góp của các thành viên. Mỗi lựa chọn công nghệ và thông số đều có hộp *Lý do* giải thích. Bộ kiểm thử tự động gồm 5 bộ với 250 phép kiểm tra chạy trên máy tính — kể cả chạy lại chính mã nguồn Arduino của hai client trên phần cứng giả lập — và đều đạt.

Từ khóa: ESP32-C3, ESP8266, FreeRTOS, ESP-IDF, RFID, UDP, mDNS, LEDC PWM, SPI DMA, ST7789, hệ thống nhúng thời gian thực.

Mục lục

Tóm tắt **i**

Danh mục từ viết tắt **ix**

1 Giới thiệu bài toán **1**
1.1 Bối cảnh . . . 1
1.2 Mục tiêu của bài toán . . . 1
1.3 Ý nghĩa . . . 1
1.4 Phạm vi . . . 2
1.5 Kết quả đạt được (tóm tắt) . . . 2
1.6 Cấu trúc báo cáo . . . 2

2 Use case của hệ thống **3**
2.1 Tác nhân . . . 3
2.2 Sơ đồ use case . . . 3
2.3 Bảng tổng hợp use case . . . 5
2.4 Đặc tả chi tiết . . . 5

3 Các thành phần hệ thống **11**
3.1 Tổng quan ba nút phần cứng . . . 11
3.2 Vi điều khiển . . . 11
3.2.1 ESP32-C3 SuperMini (máy chủ) . . . 11
3.2.2 ESP8266 NodeMCU (hai client) . . . 12
3.3 Thiết bị chấp hành và hiển thị . . . 13
3.4 Cảm biến . . . 13
3.5 Sơ đồ chân . . . 14
3.6 Thành phần phần mềm . . . 15
3.7 Các module mã nguồn của máy chủ . . . 16

4 Cài đặt môi trường, hệ điều hành và phần mềm mô phỏng **17**
4.1 Hệ điều hành đã chọn . . . 17
4.1.1 Trên thiết bị: FreeRTOS cho máy chủ, vòng lặp Arduino cho client . . . 17
4.1.2 Trên máy phát triển . . . 17
4.2 Bản quyền phần mềm . . . 17
4.3 Cài đặt từng phần mềm . . . 18
4.3.1 ESP-IDF 6.1 (cho máy chủ) . . . 18
4.3.2 Arduino IDE và ESP8266 (cho hai client) . . . 18

4.3.3 Công cụ cho giả lập và kiểm thử . . . 19
4.4 Phần mềm mô phỏng và các tính năng chính . . . 19
4.4.1 Bộ giả lập client bằng Python (`examples/`) . . . 19
4.4.2 Mô phỏng phần cứng Arduino trên máy tính (`tests/arduino/`) . . . 19

5 Thiết kế và phân tích kiến trúc 21
5.1 Kiến trúc triển khai và mạng . . . 21
5.1.1 Giao thức truyền: UDP broadcast . . . 22
5.2 Kiến trúc phần mềm máy chủ . . . 22
5.2.1 Phụ thuộc giữa các module . . . 23
5.3 Mô hình tác vụ và đồng bộ . . . 24
5.4 Luồng hoạt động . . . 26
5.4.1 Khởi động máy chủ . . . 26
5.4.2 Xử lý bản tin UDP trên máy chủ . . . 27
5.4.3 Vòng lặp của hai client . . . 28
5.4.4 Xe vào và ra bằng thẻ RFID . . . 29
5.4.5 Phát hiện cháy . . . 29
5.4.6 Trang web giám sát . . . 30
5.5 Máy trạng thái . . . 30
5.6 Giao thức và giao diện lập trình . . . 31
5.7 Thiết kế chi tiết từng thành phần . . . 32
5.7.1 Thanh chắn: PWM phần cứng và hẹn giờ một lần . . . 32
5.7.2 Màn hình: SPI DMA, vẽ theo sự kiện, ảnh động tiền xử lý . . . 33
5.7.3 Bộ kiểm tra bản tin JSON . . . 34
5.7.4 Phiên gửi xe, phí, thông báo và nhật ký . . . 34
5.7.5 Sensor Client: lọc nhiễu và gửi theo thay đổi . . . 35
5.7.6 RFID Client: hai đầu đọc trên một bus SPI . . . 35
5.7.7 Trang web . . . 36
5.8 Phân tích kiến trúc . . . 36
5.8.1 Hiệu năng và độ trễ . . . 36
5.8.2 Bộ nhớ . . . 36
5.8.3 Độ tin cậy và khả năng chịu lỗi . . . 37
5.8.4 Bảo mật . . . 37
5.8.5 Khả năng bảo trì, kiểm thử và mở rộng . . . 37
5.8.6 Tổng hợp quyết định kiến trúc . . . 38
5.9 Demo . . . 38
5.9.1 Trang web giám sát . . . 38
5.9.2 Các màn hình TFT . . . 40
5.9.3 Kịch bản demo . . . 40

6 Hướng dẫn chạy mã nguồn từng bước **41**
6.1 Xử lý sự cố thường gặp 44

7 Kiểm thử **46**
7.1 Chiến lược kiểm thử 46
7.2 Kiểm thử tự động 46
7.3 Kiểm thử tích hợp bằng công cụ giả lập 47
7.4 Kịch bản kiểm thử hệ thống trên phần cứng 47

8 Tổng kết **49**
8.1 Kết quả đạt được 49
8.2 Hạn chế 49
8.3 Hướng phát triển 50

9 Đóng góp của thành viên và phân chia điểm **51**
9.1 Phân công công việc 51
9.2 Tỉ lệ đóng góp và phân chia điểm 51

Tài liệu tham khảo **53**

Danh sách hình vẽ

2.1 Sơ đồ use case tổng quát (đường nét đứt: sự kiện theo thời gian — tự đóng thanh chắn, hết hạn thông báo) 4

5.1 Kiến trúc triển khai: RFID Client làm điểm truy cập; máy chủ, Sensor Client và điện thoại là trạm Wi-Fi 21
5.2 Kiến trúc phân lớp của firmware máy chủ 23
5.3 Đồ thị phụ thuộc giữa các module (tím: ứng dụng, xanh dương: dịch vụ, xanh lá: nghiệp vụ, cam: dữ liệu) 23
5.4 Tác vụ (xanh dương, kèm mức ưu tiên), tài nguyên dùng chung (xanh lá) và cơ chế bảo vệ; nét đứt đỏ: đánh thức tác vụ 25
5.5 Trình tự khởi động máy chủ (`main.c`) 26
5.6 Lưu đồ điều phối bản tin trong tác vụ `udp_server` 27
5.7 Vòng lặp không chặn của Sensor Client và RFID Client 28
5.8 Biểu đồ tuần tự gửi xe (trên) và lấy xe (dưới) 29
5.9 Biểu đồ tuần tự cảnh báo cháy 29
5.10 Máy trạng thái của một thẻ RFID 30
5.11 Máy trạng thái của một thanh chắn 30
5.12 Trạng thái vùng chân màn hình (footer) TFT; khi có cháy, toàn màn hình chuyển sang màn hình cảnh báo 31
5.13 Dạng xung điều khiển servo SG90 32
5.14 Chống rung: chỉ chấp nhận mức mới khi giữ ổn định 50 ms, rồi gửi ngay 35
5.15 Trang web giám sát tại `http://parking.local` (dữ liệu mẫu) 39
5.16 Bốn trạng thái màn hình TFT 240×240 (mô phỏng bố cục theo mã nguồn `lcd_display.c`) 40

7.1 Ba tầng kiểm thử: càng xuống dưới càng nhanh, tự động và chạy thường xuyên . 46

Danh sách bảng

1.1 Cấu trúc báo cáo theo yêu cầu của môn học 2

2.1 Các tác nhân của hệ thống 3
2.2 Tổng hợp 10 use case, nơi hiện thực và bộ kiểm thử tương ứng 5

3.1 Ba nút phần cứng và vai trò 11
3.2 Thông số quan trọng của ESP32-C3 SuperMini 11
3.3 So sánh phương án vi điều khiển cho máy chủ 12
3.4 Thông số quan trọng của ESP8266 12
3.5 Thiết bị chấp hành, hiển thị và thông số quan trọng 13
3.6 Cảm biến, thông số và lý do lựa chọn 13
3.7 Sơ đồ chân máy chủ ESP32-C3 14
3.8 Sơ đồ chân Sensor Client và RFID Client (ESP8266) 14
3.9 Thành phần phần mềm và vai trò 15
3.10 Module firmware máy chủ (`Server/main/`) 16

4.1 Hệ điều hành trên từng bo mạch 17
4.2 Phần mềm cần cài và giấy phép 18
4.3 Công cụ giả lập client 19

5.1 Thông số mạng 21
5.2 So sánh giao thức truyền dữ liệu cảm biến 22
5.3 Các tác vụ đang chạy trên máy chủ 24
5.4 Đặc tả trường dữ liệu và ràng buộc kiểm tra trên máy chủ 31
5.5 API HTTP của máy chủ (11 route) 32
5.6 Dữ liệu ảnh động sau tiền xử lý và nén LZSS 33
5.7 Ngân sách độ trễ của các luồng chính (ước tính từ thông số thiết kế) 36
5.8 Sử dụng bộ nhớ của firmware máy chủ (đo bằng `idf.py size`) 36
5.9 Phân tích lỗi và cách hệ thống phản ứng 37
5.10 Các quyết định kiến trúc chính 38

6.1 Tên cổng nối tiếp theo hệ điều hành 41
6.2 Lỗi thường gặp và cách khắc phục 44

7.1 Các bộ kiểm thử tự động và kết quả 46
7.2 Kịch bản kiểm thử trên phần cứng và hành vi mong đợi 47

9.1 Phân công công việc 51

9.2 Tỉ lệ đóng góp và tỉ lệ điểm 52

Danh mục mã nguồn

5.1 Bản tin `parking_update` (Sensor Client → máy chủ) 31
5.2 Bản tin `rfid_event` (RFID Client → máy chủ) 31
6.1 Cấu trúc kho mã nguồn 41
7.1 Ví dụ kiểm thử chống rung của Sensor Client 47

Danh mục từ viết tắt

Viết tắt	Ý nghĩa
ADC	Analog-to-Digital Converter — bộ chuyển đổi tương tự–số
AP	Access Point — điểm truy cập Wi-Fi
API	Application Programming Interface
DHCP	Dynamic Host Configuration Protocol — cấp địa chỉ IP tự động
DMA	Direct Memory Access — truy cập bộ nhớ trực tiếp, không qua CPU
GPIO	General Purpose Input/Output
HTTP	HyperText Transfer Protocol
IDF	IoT Development Framework (ESP-IDF)
IR	Infrared — hồng ngoại
JSON	JavaScript Object Notation
LEDC	LED Control — khối phát PWM phần cứng của ESP32
LZSS	Lempel–Ziv–Storer–Szymanski — thuật toán nén từ điển
mDNS	Multicast DNS — phân giải tên miền `.local` trong mạng cục bộ
NVS	Non-Volatile Storage — kho khóa–giá trị trên flash của ESP-IDF
OTA	Over-The-Air — nạp firmware qua mạng
PWM	Pulse Width Modulation — điều chế độ rộng xung
RFID	Radio Frequency Identification
RTOS	Real-Time Operating System
SPI	Serial Peripheral Interface
SSID	Service Set Identifier — tên mạng Wi-Fi
STA	Station — thiết bị kết nối vào một AP
TFT	Thin-Film Transistor (màn hình)
UC	Use case — ca sử dụng
UDP	User Datagram Protocol
UID	Unique Identifier — mã định danh của thẻ RFID

Chương 1. Giới thiệu bài toán

1.1 Bối cảnh

Tại các đô thị lớn như Hà Nội, số lượng ô tô và xe máy tăng nhanh hơn nhiều so với diện tích bãi đỗ. Người lái thường phải đi vòng nhiều lần để tìm chỗ trống, gây ùn tắc cục bộ và lãng phí nhiên liệu. Phần lớn bãi xe hiện nay vẫn vận hành thủ công: vé giấy, ghi biển số bằng tay, tính tiền bằng ước lượng, không biết trước còn bao nhiêu chỗ và hầu như không có cơ chế cảnh báo an toàn khi xảy ra cháy hoặc ngập nước — hai rủi ro thường gặp ở tầng hầm.

Một **bãi đỗ xe thông minh** tự động hóa các công việc này: cảm biến phát hiện chỗ trống, thẻ RFID kiểm soát ra vào và tính phí, thanh chắn tự mở/đóng, cảm biến môi trường cảnh báo sự cố và mọi thông tin được hiển thị trực tiếp tại bãi lẫn trên điện thoại của người quản lý.

1.2 Mục tiêu của bài toán

Đề tài xây dựng một mô hình bãi xe 5 chỗ, 2 cổng, đáp ứng các mục tiêu:

1. **Giám sát chỗ đỗ theo thời gian thực**: biết chính xác chỗ nào trống, chỗ nào có xe; màn hình tại bãi cập nhật trong khoảng 0,1 giây, trang web trong khoảng 1 giây.
2. **Kiểm soát ra vào bằng thẻ RFID**: thẻ quẹt ở cổng vào mở thanh chắn vào, cùng thẻ quẹt ở cổng ra mở thanh chắn ra; thanh chắn tự đóng sau 3 giây; quẹt sai cổng hoặc bãi đầy bị từ chối và thông báo rõ trên màn hình.
3. **Tính phí tự động**: 5.000 VNĐ cho mỗi giờ bắt đầu, tối thiểu 1 giờ; hiển thị hóa đơn khi xe ra.
4. **An toàn**: phát hiện lửa — bật còi, hiển thị cảnh báo, *mở cả hai thanh chắn* để sơ tán; theo dõi mực nước và nhiệt độ.
5. **Giám sát và điều khiển từ xa**: trang web tại `http://parking.local` dùng được trên mọi điện thoại, không cần cài ứng dụng, có nhật ký xe vào/ra.
6. **Hoạt động độc lập**: tự tạo mạng Wi-Fi riêng, không cần Internet hay router; tự phục hồi khi mất kết nối.

1.3 Ý nghĩa

Ý nghĩa thực tiễn. Mô hình chứng minh rằng một bãi xe nhỏ (chung cư mini, văn phòng, trường học) có thể được tự động hóa bằng linh kiện phổ thông giá rẻ. Người lái biết ngay còn chỗ hay không; người quản lý thu phí chính xác, có nhật ký xe ra vào, được cảnh báo cháy/ngập kịp thời; khi cháy, thanh chắn tự mở giúp sơ tán nhanh.

Ý nghĩa học thuật. Đây là bài toán nhúng điển hình, vận dụng gần như toàn bộ kiến thức của môn học: lập trình đa tác vụ với **hệ điều hành thời gian thực FreeRTOS**, đồng bộ giữa các tác vụ, điều khiển ngoại vi bằng phần cứng chuyên dụng (PWM, SPI có DMA, ADC, GPIO), giao tiếp không dây, quản lý bộ nhớ hạn chế (vài trăm KB RAM), xử lý dữ liệu không tin cậy từ mạng và kiểm thử phần mềm nhúng khi không có phần cứng.

1.4 Phạm vi

Hệ thống là **mô hình** quy mô phòng thí nghiệm: 5 chỗ đỗ, 2 cổng, xe mô hình, thẻ RFID MIFARE. Thanh toán điện tử, nhận dạng biển số và lưu trữ đám mây nằm ngoài phạm vi và được nêu ở phần hướng phát triển (Chương 8).

1.5 Kết quả đạt được (tóm tắt)

- Ba bo mạch hoạt động phối hợp, 10 use case (Chương 2) đều được hiện thực và có phép kiểm thử tương ứng.
- Firmware máy chủ 1,08 MB, dùng 36% RAM tĩnh; còn 29% phân vùng ứng dụng.
- 250 phép kiểm thử tự động đều đạt (Chương 7).

1.6 Cấu trúc báo cáo

Bảng 1.1: Cấu trúc báo cáo theo yêu cầu của môn học

Chương	Nội dung	Yêu cầu
1	Giới thiệu bài toán, mục tiêu, ý nghĩa	Giới thiệu (1đ)
2	10 use case: tác nhân, luồng chính, ngoại lệ, kiểm thử	Use case (10đ)
3	Các thành phần phần cứng và phần mềm, thông số quan trọng	Thành phần (2đ)
4	Cài đặt môi trường, hệ điều hành, phần mềm mô phỏng	Cài đặt (1đ)
5	Thiết kế, phân tích kiến trúc, luồng hoạt động, demo	Thiết kế (3đ)
6	Hướng dẫn chạy mã nguồn từng bước	Chạy mã nguồn (2đ)
7	Kiểm thử tự động và kịch bản kiểm thử phần cứng	Bổ trợ thiết kế/demo
8	Tổng kết, hạn chế, hướng phát triển	Kết luận (1đ)
9	Đóng góp của thành viên, phân chia điểm	Đóng góp

Chương 2. Use case của hệ thống

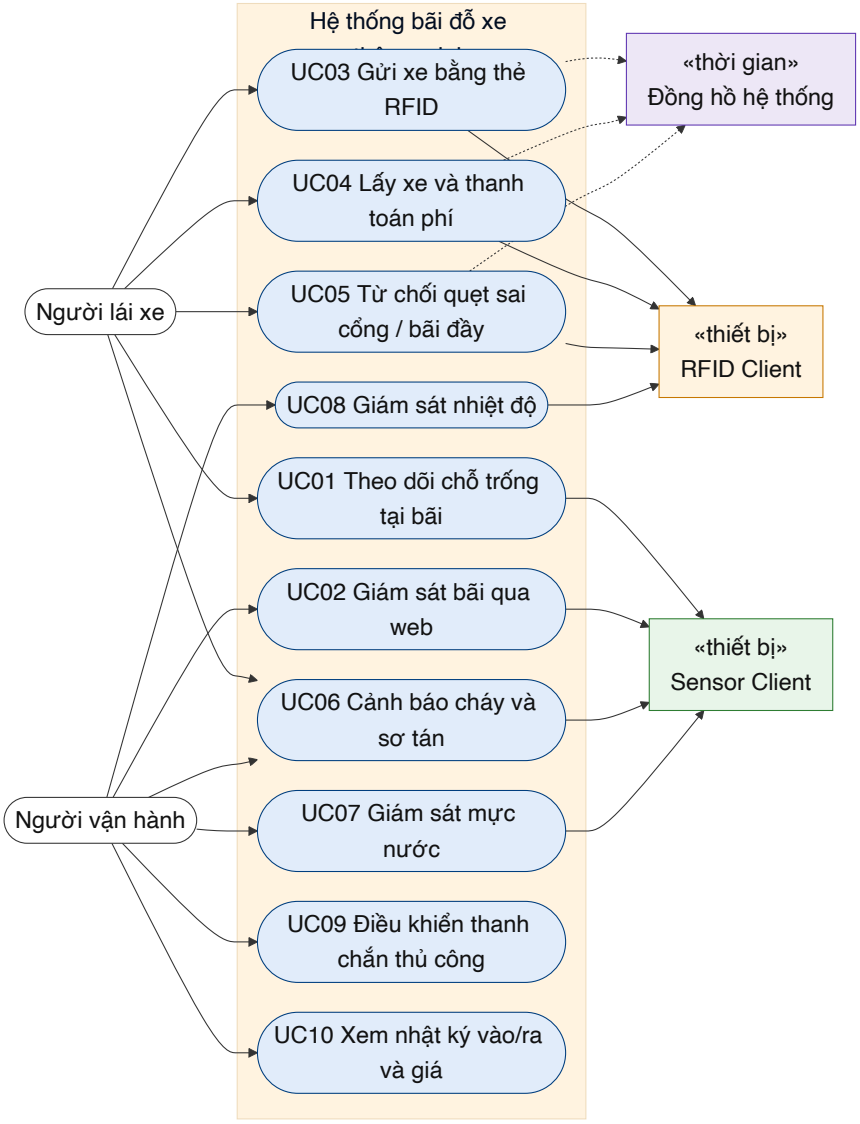
2.1 Tác nhân

Bảng 2.1: Các tác nhân của hệ thống

Tác nhân	Mô tả
Người lái xe	Người gửi/lấy xe; dùng thẻ RFID và nhìn màn hình TFT tại bãi.
Người vận hành	Bảo vệ hoặc quản lý bãi; dùng trang web trên điện thoại/máy tính để giám sát và điều khiển thanh chắn.
Sensor Client	Tác nhân thiết bị: bo ESP8266 đọc cảm biến chỗ đỗ, lửa, nước và gửi dữ liệu lên máy chủ.
RFID Client	Tác nhân thiết bị: bo ESP8266 đọc thẻ ở hai cổng và cảm biến nhiệt độ; đồng thời phát mạng Wi-Fi.
Đồng hồ hệ thống	Tác nhân thời gian: kích hoạt việc tự đóng thanh chắn, hết hạn hóa đơn/thông báo.

2.2 Sơ đồ use case

Hình 2.1 tổng hợp 10 use case và quan hệ với các tác nhân. Hai tác nhân thiết bị (bên phải) là nguồn dữ liệu; hai tác nhân con người (bên trái) là người hưởng lợi.



Hình 2.1: Sơ đồ use case tổng quát (đường nét đứt: sự kiện theo thời gian — tự đóng thanh chắn, hết hạn thông báo)

2.3 Bảng tổng hợp use case

Bảng 2.2: Tổng hợp 10 use case, nơi hiện thực và bộ kiểm thử tương ứng

Mã	Tên use case	Module chính	Kiểm thử
UC01	Theo dõi chỗ trống tại bãi	`lcd_display.c`	`sensor_client`
UC02	Giám sát bãi qua web	`http_server.c`, `index.html`	S2, S12
UC03	Gửi xe bằng thẻ RFID	`rfid_session.c`, `servo.c`	`rfid_session`, `rfid_client`
UC04	Lấy xe và thanh toán phí	`rfid_session.c`	`rfid_session`
UC05	Từ chối quẹt sai cổng / bãi đầy	`rfid_session.c`, `lcd_display.c`	`rfid_session`
UC06	Cảnh báo cháy và sơ tán	`udp_server.c`, `servo.c`, `buzzer.c`	`sensor_client`, S9
UC07	Giám sát mực nước	`SensorClient.ino`	`sensor_client`
UC08	Giám sát nhiệt độ	`RFIDClient.ino`, `parking_state.c`	`rfid_client`, `rfid_session`
UC09	Điều khiển thanh chắn thủ công	`http_server.c`, `servo.c`	S12, S13
UC10	Xem nhật ký vào/ra và giá	`rfid_session.c`, `http_server.c`	`rfid_session`

Tên tệp kiểm thử như `rfid_session` chỉ bộ kiểm thử tự động (Chương 7); S2, S9,… là kịch bản kiểm thử trên phần cứng (Bảng 7.2).

2.4 Đặc tả chi tiết

UC01 — Theo dõi chỗ trống tại bãi

Tác nhân Người lái xe (chính); Sensor Client (nguồn dữ liệu).

Mô tả Người lái nhìn màn hình TFT ở lối vào để biết số chỗ trống và vị trí từng chỗ trống.

Tiền điều kiện Sensor Client và máy chủ đã kết nối Wi-Fi `ESP32-Parking`.

Luồng chính

1. Xe đi vào/ra một ô đỗ, cảm biến IR của ô đó đổi mức logic.
2. Sensor Client lọc nhiễu: chỉ chấp nhận mức mới khi giữ ổn định 50 ms.
3. Ngay khi trạng thái đổi, Sensor Client gửi bản tin `parking_update` (UDP).

4. Máy chủ kiểm tra bản tin, cập nhật trạng thái chung và đánh thức tác vụ màn hình.
5. Màn hình vẽ lại: số chỗ trống `FREE SPACES x/5` và 5 ô S1–S5 (xanh = trống, đỏ = có xe).

Ngoại lệ 3a. Mất gói UDP: bản tin nhắc lại sau tối đa 1 s sửa lại trạng thái. 4a. Bản tin sai định dạng: bị bỏ qua, trạng thái cũ được giữ.

Hậu điều kiện Màn hình phản ánh đúng trạng thái 5 chỗ đỗ.

Thông số Debounce 50 ms; nhắc lại 1 s; độ trễ hiển thị $\approx 0{,}1$ s.

UC02 — Giám sát bãi qua trang web

Tác nhân Người vận hành.

Mô tả Xem trực quan sơ đồ bãi, số chỗ trống, cảm biến và trạng thái thanh chắn trên điện thoại.

Tiền điều kiện Thiết bị của người vận hành kết nối Wi-Fi `ESP32-Parking`.

Luồng chính
1. Người vận hành mở `http://parking.local`.
2. Thiết bị phân giải tên bằng mDNS và nhận địa chỉ IP hiện tại của máy chủ.
3. Máy chủ trả trang web nhúng trong firmware.
4. Trang web gọi `GET /api/parking` mỗi 1 s và vẽ sơ đồ 5 ô đỗ (xe màu đỏ, ô trống chữ P màu xanh).

Ngoại lệ 2a. Thiết bị không hỗ trợ `.local`: dùng địa chỉ IP in trên log máy chủ. 4a. Máy chủ mất kết nối: trang hiển thị `ESP32 / OFFLINE`.

Hậu điều kiện Người vận hành thấy trạng thái bãi với độ trễ ≤ 1 s.

UC03 — Gửi xe bằng thẻ RFID

Tác nhân Người lái xe; RFID Client; Đồng hồ hệ thống.

Mô tả Người lái quẹt thẻ ở cổng vào để được mở thanh chắn và bắt đầu tính giờ.

Tiền điều kiện Thẻ chưa có phiên gửi xe; bãi còn phiên trống (tối đa 5).

Luồng chính
1. Người lái đưa thẻ vào đầu đọc cổng vào.

2. RFID Client đọc UID, chuyển thành chuỗi hex in hoa, gửi `rfid_event` với `"gate":"in"`.
3. Máy chủ tạo phiên mới, lưu thời điểm vào, ghi nhật ký `IN`.
4. Máy chủ mở thanh chắn cổng vào (90°) và hẹn giờ đóng sau 3 s.
5. Đồng hồ hết 3 s, thanh chắn tự đóng.

Ngoại lệ 1a. Thẻ để yên trên đầu đọc: chỉ gửi một lần trong 2 s. 3a. Bãi đầy hoặc thẻ đã ở trong bãi: chuyển sang UC05. 5a. Người vận hành bấm đóng/mở trên web trước khi hết giờ: lệnh tay thắng, hủy hẹn giờ.

Hậu điều kiện Thẻ có phiên hoạt động; thanh chắn đóng lại.

Thông số Thời gian mở 3 s; chống đọc lặp 2 s; tối đa 5 phiên.

UC04 — Lấy xe và thanh toán phí

Tác nhân Người lái xe; RFID Client; Đồng hồ hệ thống.

Mô tả Người lái quẹt cùng thẻ ở cổng ra; hệ thống tính phí, hiển thị hóa đơn và mở cổng ra.

Tiền điều kiện Thẻ đang có phiên gửi xe.

Luồng chính

1. Người lái quẹt thẻ ở đầu đọc cổng ra (`"gate":"out"`).
2. Máy chủ tính thời gian gửi t và phí $\max(1, \lceil t/3600 \rceil) \times 5000$ VNĐ.
3. Máy chủ đóng phiên, ghi nhật ký `OUT` kèm thời gian và phí.
4. Màn hình hiển thị hóa đơn: UID, `PARK TIME`, `FEE VND` trong 60 s.
5. Thanh chắn cổng ra mở và tự đóng sau 3 s.

Ngoại lệ 1a. Thẻ chưa vào bãi: chuyển sang UC05.

Hậu điều kiện Phiên được giải phóng; nhật ký có bản ghi `OUT`.

Thông số 5.000 VNĐ/giờ, làm tròn lên, tối thiểu 1 giờ; hóa đơn hiển thị 60 s.

UC05 — Từ chối quẹt sai cổng / bãi đầy

Tác nhân Người lái xe; RFID Client; Đồng hồ hệ thống.

Mô tả Hệ thống từ chối các lần quẹt không hợp lệ và hướng dẫn người lái.

Tiền điều kiện Một trong ba trường hợp: (a) thẻ đã trong bãi nhưng quẹt ở cổng vào; (b) thẻ chưa vào nhưng quẹt ở cổng ra; (c) đã đủ 5 phiên mà có thẻ mới ở cổng vào.

Luồng chính

1. Máy chủ nhận `rfid_event` và xác định kết quả `INVALID_GATE` hoặc `FULL`.
2. Thanh chắn *không* mở.
3. Màn hình hiển thị thông báo đỏ trong 5 s: `WRONG GATE` kèm gợi ý `ALREADY INSIDE – USE EXIT GATE` / `NOT CHECKED IN – USE ENTRY GATE`, hoặc `PARKING FULL`.
4. Nhật ký ghi `REJECTED · WRONG GATE` hoặc `REJECTED · FULL`.

Ngoại lệ 3a. Đang hiển thị hóa đơn: thông báo hiện trước, hết 5 s hóa đơn hiện lại. 3b. Ngay sau đó có lần quẹt hợp lệ: thông báo cũ bị xóa ngay.

Hậu điều kiện Trạng thái phiên không đổi.

UC06 — Cảnh báo cháy và sơ tán

Tác nhân Sensor Client (cảm biến lửa); Người lái xe; Người vận hành.

Mô tả Khi phát hiện lửa: báo động tại chỗ, trên web và mở thanh chắn để sơ tán.

Tiền điều kiện Sensor Client đang gửi dữ liệu.

Luồng chính

1. Cảm biến lửa phát hiện ngọn lửa ổn định quá 50 ms; Sensor Client gửi ngay `"fire":true`.
2. Máy chủ bật còi (GPIO9 mức thấp).
3. Máy chủ chuyển *chế độ cháy*: mở cả hai thanh chắn và giữ mở.
4. Màn hình chuyển sang màn hình `DANGER / FIRE DETECTED / EVACUATE NOW` có ngọn lửa động.
5. Trang web hiện băng rôn đỏ `FIRE DETECTED`.

Ngoại lệ 3a. Người vận hành bấm đóng thanh chắn: bị từ chối (HTTP 500) trong chế độ cháy. 6a. Hết lửa (`"fire":false`): tắt còi, đóng cả hai thanh chắn, màn hình trở lại bình thường.

Hậu điều kiện Lối ra vào thông thoáng trong suốt thời gian có lửa.

UC07 — Giám sát mực nước

Tác nhân	Sensor Client; Người vận hành.
Mô tả	Theo dõi mực nước ngập tại bãi (mô hình tầng hầm).
Luồng chính	1. Sensor Client đọc ADC chân A0 mỗi 250 ms. 2. Quy đổi tuyến tính ADC 20–700 thành 0–40 mm (giới hạn trên 40 mm). 3. Giá trị được gửi trong `parking_update`; máy chủ kiểm tra dải 0–10.000 mm. 4. Màn hình (`WATER MM`) và web (`WATER LEVEL`) hiển thị giá trị.
Ngoại lệ	3a. Giá trị âm hoặc vượt dải: bản tin bị từ chối.
Hậu điều kiện	Mực nước hiển thị với độ phân giải 0,1 mm.

UC08 — Giám sát nhiệt độ

Tác nhân	RFID Client (DHT11); Người vận hành.
Mô tả	Theo dõi nhiệt độ môi trường tại cổng.
Luồng chính	1. RFID Client đọc DHT11 mỗi 5 s, giữ giá trị hợp lệ gần nhất. 2. Giá trị được gửi kèm mỗi `rfid_event` (`temperature_c`). 3. Máy chủ kiểm tra dải -40 đến 125 °C, cập nhật trạng thái chung. 4. Màn hình (`TEMP C`) và web (`TEMPERATURE`) hiển thị.
Ngoại lệ	1a. DHT11 đọc lỗi (NaN): giữ giá trị cũ, không gửi số sai.
Hậu điều kiện	Nhiệt độ cập nhật mỗi lần có xe quẹt thẻ.

UC09 — Điều khiển thanh chắn thủ công

Tác nhân	Người vận hành.
Mô tả	Mở/đóng từng thanh chắn hoặc cả hai (khẩn cấp) từ trang web.
Luồng chính	1. Người vận hành bấm `OPEN/CLOSE` của cổng vào, cổng ra, hoặc `EMERGENCY OPEN/CLOSE ALL`. 2. Trang web gửi yêu cầu `POST` tới `/api/barrier/<cổng>/<lệnh>` (cổng: `in`, `out`, `all`; lệnh: `open`, `close`); nút bị khóa đến khi có phản hồi. 3. Máy chủ hủy hẹn giờ tự đóng (nếu có) rồi đặt góc servo.

4. Trang web cập nhật trạng thái `OPEN`/`CLOSED`.

Ngoại lệ 3a. Đang có cháy mà yêu cầu đóng: từ chối, hiển thị lỗi.

Hậu điều kiện Thanh chắn ở đúng trạng thái người vận hành chọn.

UC10 — Xem nhật ký vào/ra và giá gửi xe

Tác nhân Người vận hành.

Mô tả Biết ai đang trong bãi, đã gửi bao lâu, phí tạm tính, và 10 sự kiện gần nhất.

Luồng chính

1. Trang web gọi `GET /api/rfid` mỗi 2 s.
2. Máy chủ trả giá mỗi giờ, danh sách xe trong bãi (UID, giờ vào, phí tạm tính) và 10 sự kiện mới nhất (vào, ra kèm thời lượng và phí, bị từ chối).
3. Trình duyệt đổi thời gian "giây kể từ khi khởi động" thành giờ thực theo đồng hồ điện thoại.

Ngoại lệ 2a. Máy chủ khởi động lại: nhật ký và phiên được làm mới (lưu trong RAM).

Hậu điều kiện Người vận hành đối soát được tiền thu và xe trong bãi.

Chương 3. Các thành phần hệ thống

Chương này liệt kê mọi thành phần phần cứng và phần mềm của hệ thống, kèm các thông số quan trọng quyết định thiết kế. Lý do chọn từng thành phần được trình bày trong các hộp *Lý do*; phần kiến trúc tổng thể nằm ở Chương 5.

3.1 Tổng quan ba nút phần cứng

Bảng 3.1: Ba nút phần cứng và vai trò

Nút	Thiết bị gắn kèm và nhiệm vụ	Mã nguồn
Máy chủ (ESP32-C3)	TFT ST7789, 2 servo SG90, còi. Nhận UDP, quản lý phiên RFID, tính phí, điều khiển thanh chắn, web server, mDNS	`Server/main/`
Sensor Client (ESP8266)	5 cảm biến IR, cảm biến lửa, cảm biến mực nước. Lọc nhiễu và gửi trạng thái	`SensorClient/`
RFID Client (ESP8266)	2 đầu đọc MFRC522, DHT11. Phát Wi-Fi, gửi sự kiện quẹt thẻ	`RFIDClient/`

3.2 Vi điều khiển

3.2.1 ESP32-C3 SuperMini (máy chủ)

Bảng 3.2: Thông số quan trọng của ESP32-C3 SuperMini

CPU	RISC-V 32 bit, 1 nhân, chạy ở 160 MHz
Bộ nhớ	400 KB SRAM (ứng dụng dùng được $\approx$ 321 KB DRAM), 384 KB ROM
Flash	Flash ngoài, cấu hình 2 MB; phân vùng ứng dụng 1,5 MB
Không dây	Wi-Fi 802.11 b/g/n 2,4 GHz, Bluetooth LE 5
Ngoại vi dùng tới	SPI2 có GDMA, LEDC (PWM 14 bit), GPIO, bộ định thời `esp_timer` 64 bit
Nạp/gỡ lỗi	USB Serial/JTAG tích hợp (không cần chip USB-UART, không cần driver)
Kích thước	22×18 mm

Bảng 3.3: So sánh phương án vi điều khiển cho máy chủ

Tiêu chí	ESP32-C3	ESP32	ESP8266	Arduino Uno
CPU	RISC-V 160 MHz	Xtensa 2 nhân 240 MHz	Xtensa 80/160 MHz	AVR 16 MHz
SRAM	400 KB	520 KB	∼80 KB	2 KB
Wi-Fi	Có	Có	Có	Không
PWM phần cứng	LEDC 14 bit	LEDC	PWM mềm	8 bit
SPI có DMA	Có	Có	Hạn chế	Không
RTOS chính thức	FreeRTOS	FreeRTOS	Hạn chế	Không

Lý do lựa chọn. Máy chủ phải *đồng thời* giữ Wi-Fi, chạy web server và UDP server, vẽ màn hình 240×240 màu 16 bit và phát PWM cho servo. Chỉ riêng ảnh màn hình đã cần $240 \times 240 \times 2 = 115\,200$ byte RAM — lớn hơn toàn bộ RAM của ESP8266 hay Arduino. ESP32-C3 có đủ RAM, có DMA cho SPI (CPU không phải chờ khi đẩy ảnh), có PWM phần cứng (servo không rung khi CPU bận mạng) và được hỗ trợ chính thức bởi ESP-IDF/FreeRTOS. So với ESP32 bản gốc, C3 rẻ và nhỏ hơn; một nhân 160 MHz đã dư năng lực cho bài toán này.

3.2.2 ESP8266 NodeMCU (hai client)

Bảng 3.4: Thông số quan trọng của ESP8266

CPU	Tensilica L106 32 bit, 80 MHz
Bộ nhớ	∼80 KB RAM dữ liệu cho người dùng, flash 4 MB trên bo NodeMCU
Ngoại vi dùng tới	1 kênh ADC 10 bit (A0), SPI phần cứng (HSPI), GPIO
Không dây	Wi-Fi 802.11 b/g/n, chạy được chế độ STA hoặc AP

Lý do lựa chọn. Nút cảm biến chỉ đọc vài chân số, một chân ADC, giao tiếp SPI với RC522 và gửi gói UDP nhỏ — nhu cầu tính toán rất thấp. ESP8266 là bo Wi-Fi rẻ và phổ biến nhất, có ADC, có hệ sinh thái thư viện Arduino lớn (MFRC522, DHT, ArduinoOTA). Tách ra hai bo vì máy chủ không đủ chân: màn hình, servo và còi đã dùng 9 chân trong khoảng 13 chân khả dụng của ESP32-C3 SuperMini.

3.3 Thiết bị chấp hành và hiển thị

Bảng 3.5: Thiết bị chấp hành, hiển thị và thông số quan trọng

Thiết bị	**Thông số quan trọng**
TFT ST7789 1,54”	240×240, IPS, 262K màu (dùng RGB565 16 bit), SPI chỉ ghi, chu kỳ ghi tối thiểu $\approx$ 16 ns ($\approx$ 62 MHz); hệ thống chạy 40 MHz
Servo SG90 $\times 2$	4,8–6 V; mô-men $\approx$ 1,8 kg·cm; tốc độ $\approx$ 0,1 s/60°; điều khiển PWM 50 Hz, xung $\approx$ 500–2400 µs cho 0–180°
Còi (module)	Tích cực mức thấp: chân tín hiệu mức 0 thì kêu

3.4 Cảm biến

Bảng 3.6: Cảm biến, thông số và lý do lựa chọn

Cảm biến	**Thông số**	**Lý do lựa chọn**
IR phản xạ (FC-51) $\times 5$	2–30 cm, so sánh LM393 có biến trở, ngõ ra số tích cực mức thấp	Rẻ, phát hiện vật ở cự ly vài cm hợp với xe mô hình, chỉ tốn 1 chân. Siêu âm chính xác hơn nhưng tốn 2 chân/cảm biến và phải đo thời gian.
Cảm biến lửa	Bước sóng 760–1100 nm, góc $\approx$ 60°, ngõ ra số có ngưỡng	Phản ứng nhanh hơn cảm biến khói/nhiệt; ngõ ra số sẵn ngưỡng.
Mực nước (điện trở)	Vùng đo $\approx$ 40 mm, ngõ ra tương tự	Đơn giản, chỉ cần 1 chân ADC, hợp với mô hình tầng hầm.
RFID MFRC522 $\times 2$	13,56 MHz, ISO 14443A (MIFARE), SPI, 3,3 V, đọc vài cm	Đọc được thẻ phổ biến (thẻ sinh viên, móc khóa), thư viện ổn định, giá rẻ.
DHT11	0–50 °C (± 2 °C), 20–90%RH, đọc tối đa 1 lần/giây	Đủ cho giám sát môi trường, chỉ cần 1 chân.

3.5 Sơ đồ chân

Bảng 3.7: Sơ đồ chân máy chủ ESP32-C3

Tín hiệu	GPIO	Ghi chú
TFT SCK / MOSI	4 / 6	SPI2 qua GPIO matrix; màn hình không cần MISO
TFT DC / RST / CS	7 / 3 / 10	Chọn lệnh–dữ liệu, reset, chọn chip
TFT đèn nền	5	Tắt khi khởi tạo để không thấy rác, bật sau khi sẵn sàng
Servo cổng vào / ra	0 / 1	Chân thường (bo không dùng thạch anh 32 kHz)
Còi	9	Tích cực mức thấp; GPIO9 là chân strapping có kéo lên, trùng với trạng thái "im lặng"

Bảng 3.8: Sơ đồ chân Sensor Client và RFID Client (ESP8266)

Tín hiệu	GPIO	Ghi chú
Sensor Client		
Ô đỗ S1–S5	16, 4, 14, 13, 12	Tránh chân strapping (0, 2, 15) và UART (1, 3)
Cảm biến lửa (DO)	5	Module dùng trong nhóm cho mức cao khi có lửa
Mực nước	A0	Chân ADC duy nhất của ESP8266
RFID Client		
SPI SCK / MOSI / MISO	14 / 13 / 12	SPI phần cứng, dùng chung cho 2 đầu đọc
SS đầu đọc cổng vào	5 (D1)	Mỗi thiết bị SPI cần chân chọn chip riêng
SS đầu đọc cổng ra	4 (D2)	
RST chung	16 (D0)	Dùng chung để tiết kiệm chân
DHT11	2 (D4)	GPIO2 cần mức cao khi khởi động, DHT11 có điện trở kéo lên

Nguồn điện. Servo SG90 có dòng khởi động vài trăm mA mỗi con; nếu lấy từ chân 3V3 của bo, sụt áp làm ESP32-C3 reset. Vì vậy servo dùng nguồn 5 V riêng, **nối chung GND** với ESP32-C3 để tín hiệu PWM có cùng mức tham chiếu.

3.6 Thành phần phần mềm

Bảng 3.9: Thành phần phần mềm và vai trò

Thành phần	**Phiên bản**	**Vai trò**
ESP-IDF	6.1	Framework chính thức cho ESP32-C3: driver, Wi-Fi, lwIP, HTTP server
FreeRTOS	đi kèm ESP-IDF	Hệ điều hành thời gian thực: tác vụ, ưu tiên, mutex, thông báo tác vụ
`esp_lcd` + SPI master	ESP-IDF	Driver ST7789, truyền ảnh qua DMA
LEDC, `esp_timer`	ESP-IDF	PWM servo; hẹn giờ đóng thanh chắn
`esp_http_server`	ESP-IDF	Phục vụ trang web và API JSON
`espressif/mdns`	1.14.1	Tên `parking.local` trong mạng cục bộ
Arduino core ESP8266	3.x	Nền tảng cho hai client
MFRC522, DHT sensor library	thư viện Arduino	Đọc RFID, đọc DHT11
ArduinoOTA	đi kèm core	Nạp firmware client qua Wi-Fi
HTML/CSS/JavaScript	thuần	Trang web, không framework, không CDN

3.7 Các module mã nguồn của máy chủ

Bảng 3.10: Module firmware máy chủ (`Server/main/`)

Tệp	Dòng	Trách nhiệm
`main.c`	39	Khởi tạo NVS, màn hình, còi, servo, mạng theo thứ tự
`parking/parking_state.c`	48	Trạng thái bãi dùng chung (critical section)
`parking/json_scan.c`	81	Hàm đọc JSON dùng chung, không cấp phát động
`parking/parking_update.c`	74	Kiểm tra bản tin `parking_update`
`parking/rfid_session.c`	256	Bản tin RFID, phiên gửi xe, phí, hóa đơn, thông báo, nhật ký
`barrier/servo.c`	230	PWM servo, hẹn giờ tự đóng, chế độ cháy
`buzzer/buzzer.c`	29	Còi tích cực mức thấp
`display/lcd_display.c`	623	Giao diện TFT, màn hình cháy, ảnh động
`display/*_gif.c`	dữ liệu	Khung ảnh động đã nén LZSS
`network/wifi_sta.c`	134	Kết nối Wi-Fi, tự kết nối lại
`network/network_service.c`	81	Khởi động mạng, mDNS, cập nhật IP cho mDNS
`network/udp/udp_server.c`	138	Nhận và điều phối bản tin UDP
`network/http/http_server.c`	195	11 route HTTP, API JSON

Chương 4. Cài đặt môi trường, hệ điều hành và phần mềm mô phỏng

4.1 Hệ điều hành đã chọn

4.1.1 Trên thiết bị: FreeRTOS cho máy chủ, vòng lặp Arduino cho client

Bảng 4.1: Hệ điều hành trên từng bo mạch

Bo mạch	Hệ điều hành / mô hình chạy	Tính năng được dùng
ESP32-C3 (máy chủ)	FreeRTOS (bản Espressif, đi kèm ESP-IDF 6.1), tick 100 Hz	Tác vụ có ưu tiên, mutex, critical section, thông báo tác vụ, hẹn giờ `esp_timer`
ESP8266 (client)	Không dùng RTOS: một vòng lặp `loop()` không chặn trên Arduino core	Đếm thời gian bằng `millis()`, mọi việc chia nhỏ để Wi-Fi luôn được phục vụ

Tại sao máy chủ cần RTOS còn client thì không. Máy chủ có nhiều việc chạy song song với chu kỳ khác nhau: nhận UDP bất kỳ lúc nào, phục vụ HTTP, vẽ màn hình, chạy ảnh động 10 khung/giây, đóng thanh chắn đúng giờ. Viết tất cả trong một vòng lặp buộc phải chia nhỏ mọi việc bằng tay, và chỉ một thao tác chậm (vẽ màn hình ≈ 23 ms) cũng làm trễ việc khác. FreeRTOS cho phép mỗi việc là một tác vụ có ưu tiên riêng: việc quan trọng (thanh chắn, nhận bản tin) luôn chạy trước việc trang trí (ảnh động). Client chỉ làm tuần tự vài việc ngắn (đọc cảm biến, gửi gói tin), một vòng lặp không chặn là đủ và đơn giản hơn.

4.1.2 Trên máy phát triển

Toàn bộ công cụ chạy trên **Windows 10/11, macOS hoặc Linux**. Nhóm đã phát triển trên macOS và Windows. Các lệnh trong báo cáo dùng được trên cả ba hệ điều hành, khác biệt duy nhất là tên cổng nối tiếp (Bảng 6.1).

4.2 Bản quyền phần mềm

Mọi phần mềm dùng trong đề tài đều **miễn phí và mã nguồn mở**, nên không cần bản dùng thử (trial) hay giấy phép trả phí:

Bảng 4.2: Phần mềm cần cài và giấy phép

Phần mềm	**Dùng để**	**Giấy phép**
ESP-IDF 6.1 (qua ESP-IDF Installation Manager)	Biên dịch, nạp firmware máy chủ	Apache 2.0
Visual Studio Code + ESP-IDF extension (tùy chọn)	Soạn thảo, build/flash bằng giao diện	MIT / Apache 2.0
Arduino IDE 2.x	Biên dịch, nạp hai client	AGPL-3.0
Arduino core ESP8266	Hỗ trợ bo ESP8266	LGPL-2.1
MFRC522, DHT sensor library	Thư viện thiết bị	Unlicense / MIT
Python 3	Chạy công cụ giả lập client	PSF
Trình biên dịch C/C++ (clang hoặc gcc)	Chạy bộ kiểm thử trên máy tính	Apache 2.0 / GPL
Git	Lấy mã nguồn	GPL-2.0

4.3 Cài đặt từng phần mềm

4.3.1 ESP-IDF 6.1 (cho máy chủ)

1. Tải **ESP-IDF Installation Manager (EIM)** tại `https://dl.espressif.com/dl/eim/` và cài như ứng dụng thông thường.
2. Mở EIM, chọn **ESP-IDF v6.1**, mục tiêu **esp32c3**, bấm cài đặt. EIM tự tải trình biên dịch RISC-V, Python và các công cụ.
3. Mở terminal ESP-IDF. Windows: shortcut "ESP-IDF 6.1 PowerShell" do EIM tạo. macOS/Linux: nạp script kích hoạt do EIM tạo, ví dụ:

```
source ~/.espressif/tools/activate_idf_v6.1.sh
```

4. Kiểm tra: `idf.py --version` phải in `ESP-IDF v6.1`.

ESP32-C3 SuperMini dùng USB Serial/JTAG tích hợp nên **không cần cài driver**.

4.3.2 Arduino IDE và ESP8266 (cho hai client)

1. Cài Arduino IDE 2.x từ `https://www.arduino.cc/en/software`.
2. *File* → *Preferences* → *Additional boards manager URLs*, thêm: `https://arduino.esp8266.com/stable/package_esp8266com_index.json`
3. *Tools* → *Board* → *Boards Manager*, tìm **esp8266**, cài.

4. *Tools* → *Manage Libraries*, cài **MFRC522** (GithubCommunity) và **DHT sensor library** (Adafruit, chấp nhận cài kèm Adafruit Unified Sensor).
5. Bo NodeMCU dùng chip USB-UART CH340 hoặc CP2102: nếu máy không nhận cổng, cài driver tương ứng của nhà sản xuất chip.

4.3.3 Công cụ cho giả lập và kiểm thử

- **Python 3.8+**: có sẵn trên macOS/Linux; Windows tải tại python.org. Không cần thư viện ngoài.
- **Trình biên dịch C/C++**: macOS `xcode-select --install`; Ubuntu `sudo apt install build-essential`; Windows dùng MSYS2/MinGW hoặc WSL.

4.4 Phần mềm mô phỏng và các tính năng chính

Đề tài dùng hai công cụ mô phỏng do nhóm tự viết, giúp phát triển và kiểm tra hệ thống khi chưa có (hoặc không mang theo) đủ phần cứng.

4.4.1 Bộ giả lập client bằng Python (`examples/`)

Bảng 4.3: Công cụ giả lập client

Công cụ	Tính năng
`udp_sender.py`	Giả lập Sensor Client: gửi một bản tin `parking_update` (5 ô đỗ, mực nước, cháy) tới máy chủ thật qua UDP broadcast
`udp_rfid_sender.py`	Giả lập RFID Client: gửi `rfid_event` với UID bất kỳ và cổng `--gate in|out`; chạy hai lần để giả lập vào rồi ra
`udp_sender_espidf.c`	Ví dụ gửi bản tin từ một bo ESP-IDF khác

Máy tính chỉ cần kết nối Wi-Fi `ESP32-Parking`; máy chủ không phân biệt bản tin đến từ ESP8266 thật hay từ script, nên toàn bộ màn hình, thanh chắn, còi, trang web và nhật ký đều được kiểm tra như thật.

4.4.2 Mô phỏng phần cứng Arduino trên máy tính (`tests/arduino/`)

Đây là lớp giả lập các thư viện Arduino/ESP8266: đồng hồ `millis()` điều khiển được, mức logic của từng chân, giá trị ADC, Wi-Fi bật/tắt, gói UDP được ghi lại, đầu đọc RFID có thể "đặt thẻ" và DHT11 trả nhiệt độ tùy ý. Nhờ vậy *chính mã nguồn* `SensorClient.ino` và `RFIDClient.ino` được biên dịch và chạy trên máy tính, mỗi bản tin chúng gửi được đưa thẳng

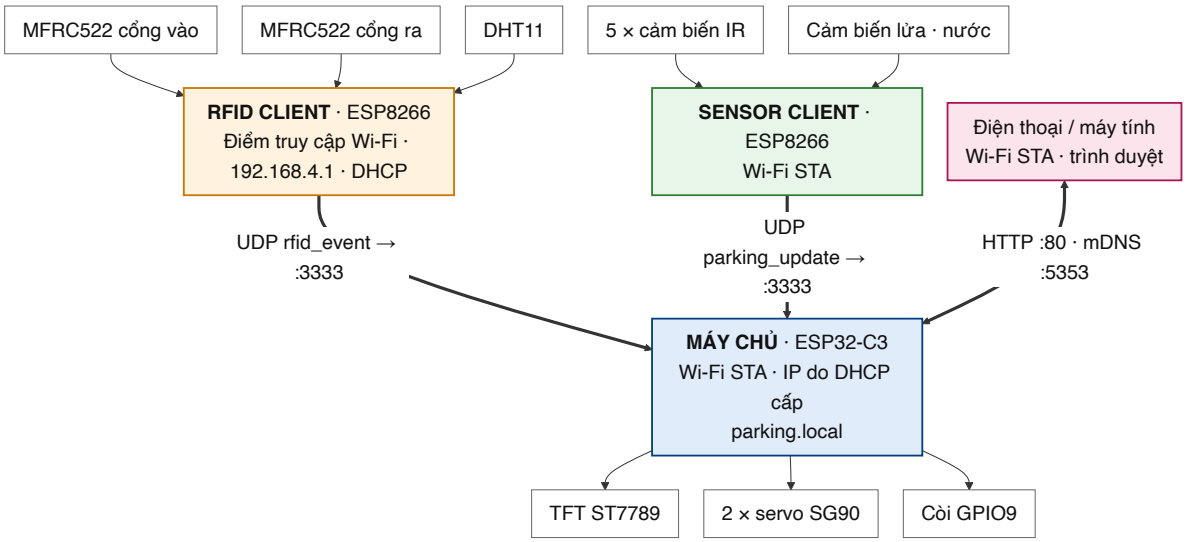
vào bộ đọc JSON thật của máy chủ để kiểm tra hai bên "nói cùng một ngôn ngữ". Chi tiết ở Chương 7.

> **Tại sao tự viết công cụ mô phỏng thay vì dùng trình mô phỏng có sẵn**. Các trình mô phỏng vi điều khiển trực tuyến hỗ trợ ESP32-C3 nhưng không mô phỏng được việc ba bo mạch nói chuyện với nhau qua Wi-Fi, và không có mô hình cho cả MFRC522 lẫn module lửa/nước cụ thể của nhóm. Giả lập ở mức *giao thức* (script Python gửi bản tin thật) và ở mức *thư viện* (đồng hồ, chân, UDP giả) kiểm tra đúng phần logic nhóm tự viết, chạy trong vài giây, lặp lại được và không phụ thuộc dịch vụ bên ngoài.

Chương 5. Thiết kế và phân tích kiến trúc

Chương này đi từ tổng thể đến chi tiết: kiến trúc triển khai và mạng (§5.1), kiến trúc phần mềm máy chủ (§5.2), mô hình tác vụ và đồng bộ (§5.3), các luồng hoạt động (§5.4), máy trạng thái (§5.5), giao thức (§5.6), thiết kế chi tiết từng module (§5.7), phân tích chất lượng kiến trúc (§5.8) và demo (§5.9).

5.1 Kiến trúc triển khai và mạng



Hình 5.1: Kiến trúc triển khai: RFID Client làm điểm truy cập; máy chủ, Sensor Client và điện thoại là trạm Wi-Fi

Hình 5.1 cho thấy cấu trúc mạng hình sao quanh RFID Client. Bảng 5.1 liệt kê thông số mạng. Hai client gửi tới địa chỉ *broadcast* `192.168.4.255` nên không cần biết IP máy chủ; người dùng truy cập bằng tên `parking.local` nên cũng không cần biết IP.

Bảng 5.1: Thông số mạng

SSID / mật khẩu	`ESP32-Parking` / `12345678`, WPA2-PSK, kênh 1, tối đa 4 trạm
Điểm truy cập	RFID Client, `192.168.4.1/24`, cấp IP bằng DHCP
Máy chủ	Trạm Wi-Fi, IP do DHCP cấp, tên `parking.local`, dịch vụ `_http._tcp` cổng 80
UDP	Máy chủ nghe cổng 3333; client gửi tới `192.168.4.255:3333`
HTTP	Cổng 80: trang web, `/api/parking`, `/api/barrier`, `/api/rfid`

Tại sao RFID Client làm điểm truy cập và máy chủ dùng mDNS. Đặt điểm truy cập trên một bo ESP8266 giải phóng ESP32-C3 khỏi việc quản lý trạm (bộ đệm cho từng trạm, DHCP server), dành RAM cho màn hình. Hệ thống vẫn hoạt động độc lập, không cần router hay Internet. Đổi lại, IP máy chủ do DHCP cấp và có thể thay đổi; mDNS giải quyết bằng một tên cố định `parking.local`. Máy chủ còn *chủ động thông báo lại* tên này mỗi khi nhận địa chỉ mới (sự kiện `IP_EVENT_STA_GOT_IP`), kể cả khi DHCP đổi địa chỉ mà không mất kết nối, nên các thiết bị khác không bao giờ giữ một IP cũ.

5.1.1 Giao thức truyền: UDP broadcast

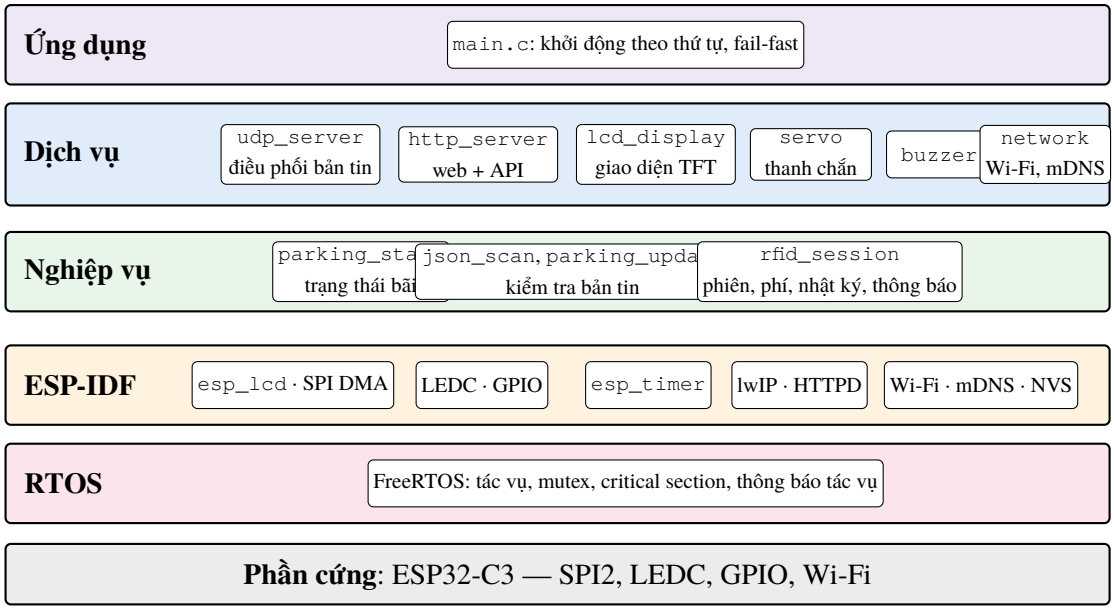
Bảng 5.2: So sánh giao thức truyền dữ liệu cảm biến

	UDP	**HTTP/TCP**	**MQTT**
Thiết lập kết nối	Không	Bắt tay 3 bước	Cần broker
Overhead/bản tin	8 byte	Vài trăm byte	2+ byte trên TCP
Bộ nhớ máy chủ	1 socket, đệm 256 B	Socket + đệm mỗi kết nối	Thêm broker
Khi mất gói	Mất, bản tin sau bù lại	Tự truyền lại	Có QoS

Lý do lựa chọn. Sensor Client gửi **toàn bộ trạng thái** (không phải "sự kiện thay đổi"), và nhắc lại mỗi giây. Mất một gói chỉ làm trễ tối đa 1 s, không làm sai trạng thái lâu dài, nên độ tin cậy của TCP là không cần thiết, trong khi chi phí của nó (bắt tay, socket, bộ đệm) đáng kể trên vi điều khiển. Gửi broadcast giúp client không cần biết IP máy chủ và cho phép máy tính trong mạng "nghe" để gỡ lỗi. JSON được chọn vì đọc được trong log và dễ sinh từ `snprintf`/Python; bản tin dài 70–110 byte, nằm gọn trong một gói UDP.

5.2 Kiến trúc phần mềm máy chủ

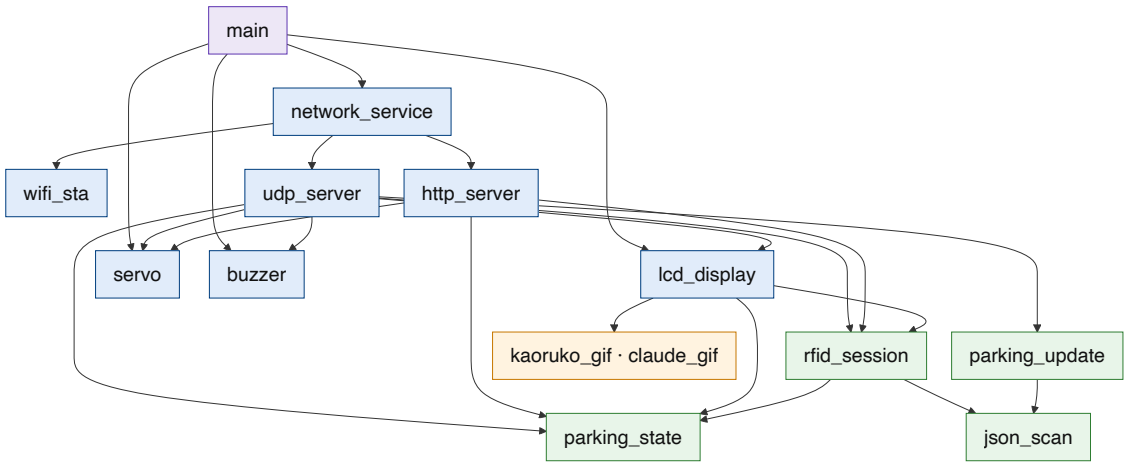
Firmware máy chủ được tổ chức theo **kiến trúc phân lớp** (Hình 5.2): mỗi lớp chỉ gọi lớp dưới nó. Lớp *nghiệp vụ* (`parking/`) không phụ thuộc phần cứng nên chạy được cả trên máy tính để kiểm thử.



Hình 5.2: Kiến trúc phân lớp của firmware máy chủ

5.2.1 Phụ thuộc giữa các module

Hình 5.3 là đồ thị phụ thuộc: mũi tên từ A đến B nghĩa là A gọi B. Mọi mũi tên đều đi xuống lớp thấp hơn, hoặc từ `udp_server` sang các thiết bị cùng lớp dịch vụ, nên không có vòng phụ thuộc.



Hình 5.3: Đồ thị phụ thuộc giữa các module (tím: ứng dụng, xanh dương: dịch vụ, xanh lá: nghiệp vụ, cam: dữ liệu)

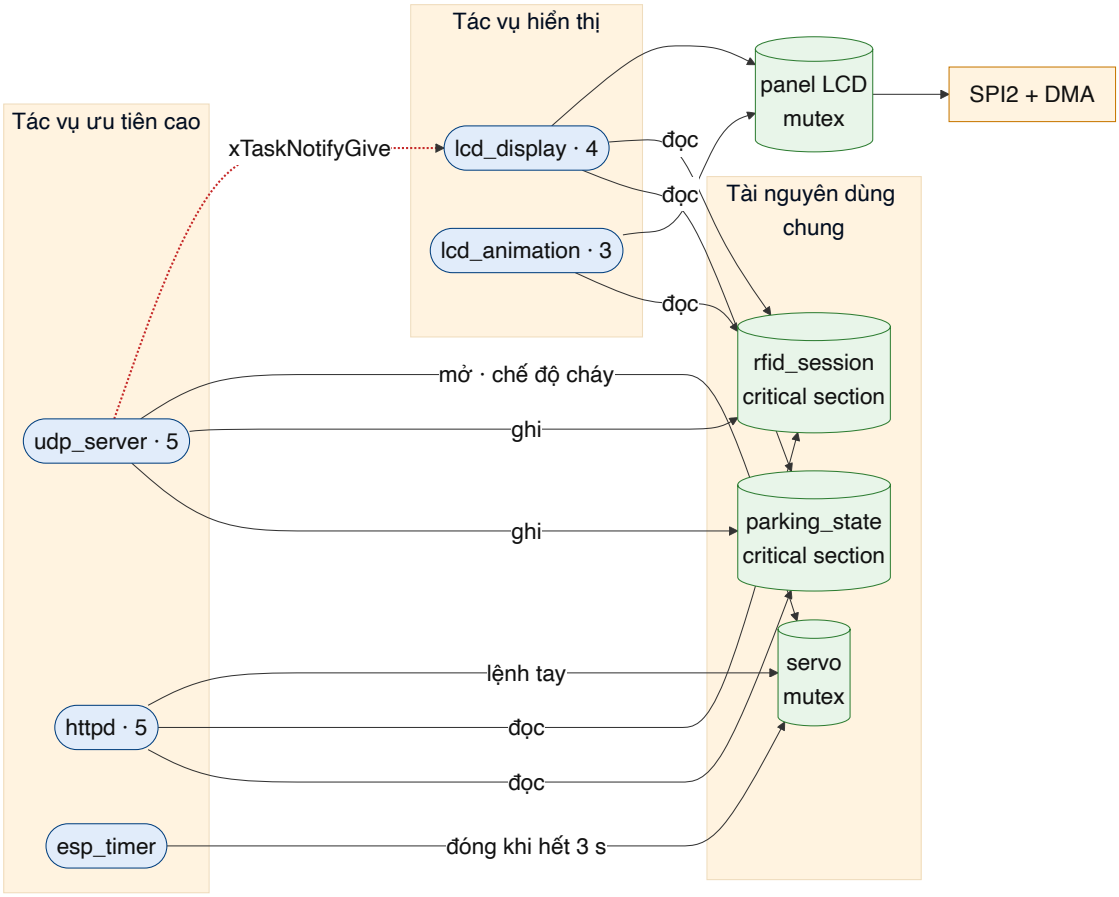
Tại sao phân lớp như vậy. (1) *Kiểm thử được*: `parking_state`, `json_scan`, `parking_update`, `rfid_session` chỉ cần hai macro critical section và hàm đọc đồng hồ — thay bằng hai tệp header giả là chạy được trên máy tính. (2) *Thay đổi cục bộ*: đổi màn hình khác chỉ sửa `lcd_display`; đổi giao thức mạng chỉ sửa `udp_server`. (3) *Không vòng phụ thuộc* (Hình 5.3): `udp_server` là nơi duy nhất "điều phối" — nhận bản tin, gọi nghiệp vụ, rồi ra

lệnh cho servo, còi và đánh thức màn hình.

5.3 Mô hình tác vụ và đồng bộ

Bảng 5.3: Các tác vụ đang chạy trên máy chủ

Tác vụ	Ưu tiên	Stack	Đánh thức bởi	Nhiệm vụ
`udp_server`	5	4096 B	gói UDP đến	Kiểm tra bản tin, cập nhật trạng thái, ra lệnh servo/còi/màn hình
`httpd`	5	4096 B	request HTTP	Trang web, API
`esp_timer` (hệ thống)	cao	3584 B	hẹn giờ	Gọi hàm đóng thanh chắn khi hết 3 s
`lcd_display`	4	4096 B	thông báo tác vụ / 1 s / 120 ms	Vẽ lại màn hình khi dữ liệu đổi
`lcd_animation`	3	3072 B	mỗi khung (100 ms)	Ảnh động trang trí
`mdns`	1	4096 B	gói mDNS	Trả lời `parking.local`



Hình 5.4: Tác vụ (xanh dương, kèm mức ưu tiên), tài nguyên dùng chung (xanh lá) và cơ chế bảo vệ; nét đứt đỏ: đánh thức tác vụ

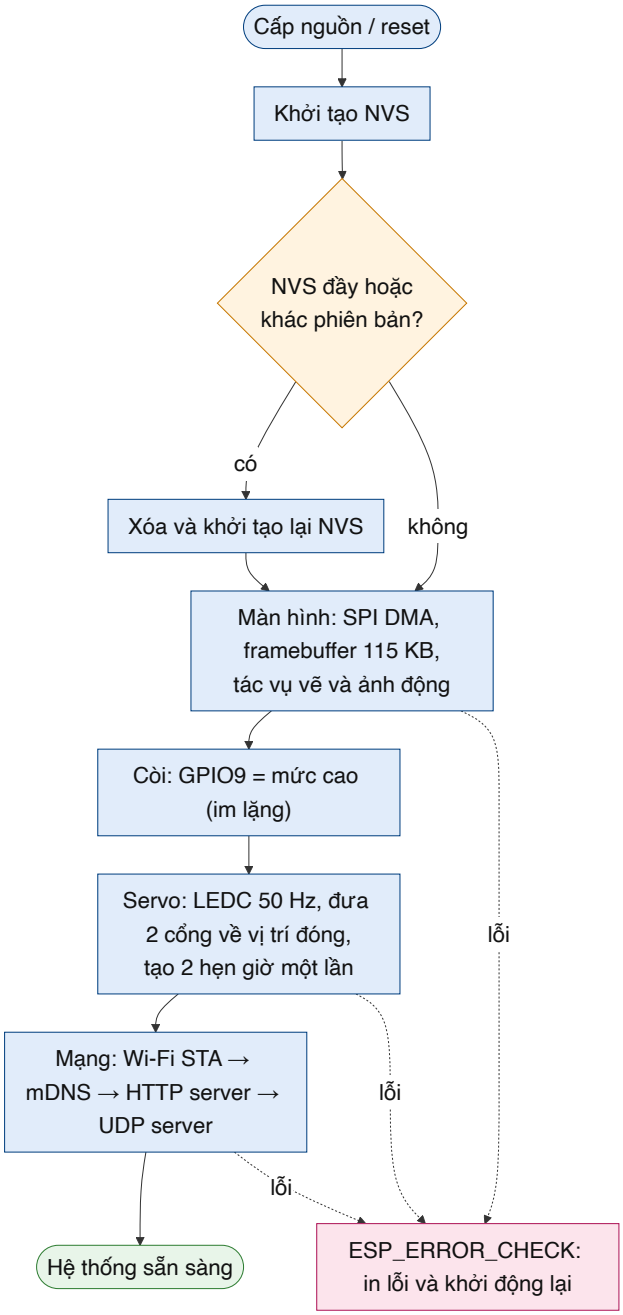
Hệ thống dùng **ba cơ chế đồng bộ** cho ba loại tình huống khác nhau:

- **Critical section** (`portENTER_CRITICAL`) cho `parking_state` và `rfid_session`: vùng bảo vệ chỉ là vài lệnh sao chép cấu trúc nhỏ (vài micro giây). Rẻ hơn mutex; người đọc luôn nhận một bản sao nhất quán.
- **Mutex** cho servo và panel LCD: vùng bảo vệ dài hơn (gửi dữ liệu SPI, ghi thanh ghi LEDC) và có thể chờ; mutex không tắt ngắt, cho phép tác vụ khác chạy và có kế thừa ưu tiên chống đảo ưu tiên.
- **Thông báo tác vụ** (`xTaskNotifyGive`/`ulTaskNotifyTake`) để `udp_server` đánh thức `lcd_display` ngay khi có dữ liệu mới — nhẹ hơn semaphore (không cấp phát đối tượng) và thay cho việc hỏi vòng 500 ms trước đây.

> **Tại sao đặt ưu tiên như vậy**. Ưu tiên phản ánh mức độ quan trọng với người dùng: nhận bản tin và điều khiển thanh chắn (5) liên quan trực tiếp đến xe đang chờ ở cổng; màn hình thông tin (4) quan trọng hơn ảnh động trang trí (3); mDNS (1) chỉ trả lời truy vấn tên, chậm vài mili giây không ảnh hưởng. Khi CPU bận, ảnh động chậm đi chứ thông tin không bị trễ.

5.4 Luồng hoạt động

5.4.1 Khởi động máy chủ

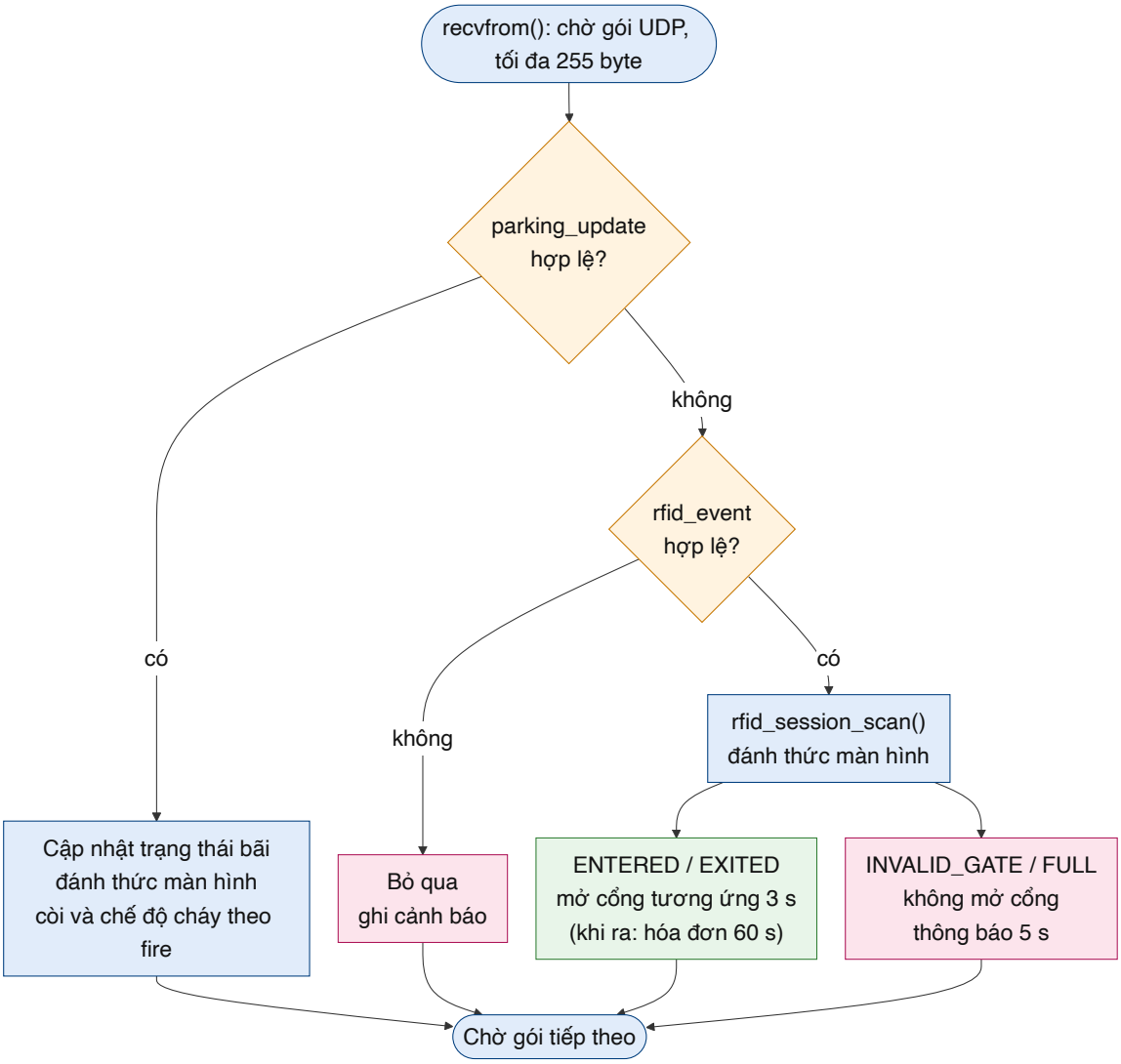


Hình 5.5: Trình tự khởi động máy chủ (`main.c`)

Tại sao thứ tự khởi động như vậy. **NVS trước tiên** vì Wi-Fi đọc dữ liệu hiệu chuẩn từ NVS. **Màn hình trước Wi-Fi**: framebuffer cần một khối bộ nhớ *liền mạch* 115 KB có khả năng DMA; Wi-Fi cấp phát nhiều khối nhỏ làm phân mảnh heap, nên phải lấy khối lớn trước. **Còi và servo trước mạng**: đưa còi về im lặng và thanh chắn về vị trí đóng trước khi

có bất kỳ lệnh nào từ mạng. **Mạng sau cùng**: chỉ nhận lệnh khi mọi thiết bị đã sẵn sàng. `ESP_ERROR_CHECK` áp dụng nguyên tắc *fail-fast*: lỗi khởi tạo dẫn tới khởi động lại thay vì chạy ở trạng thái nửa vời.

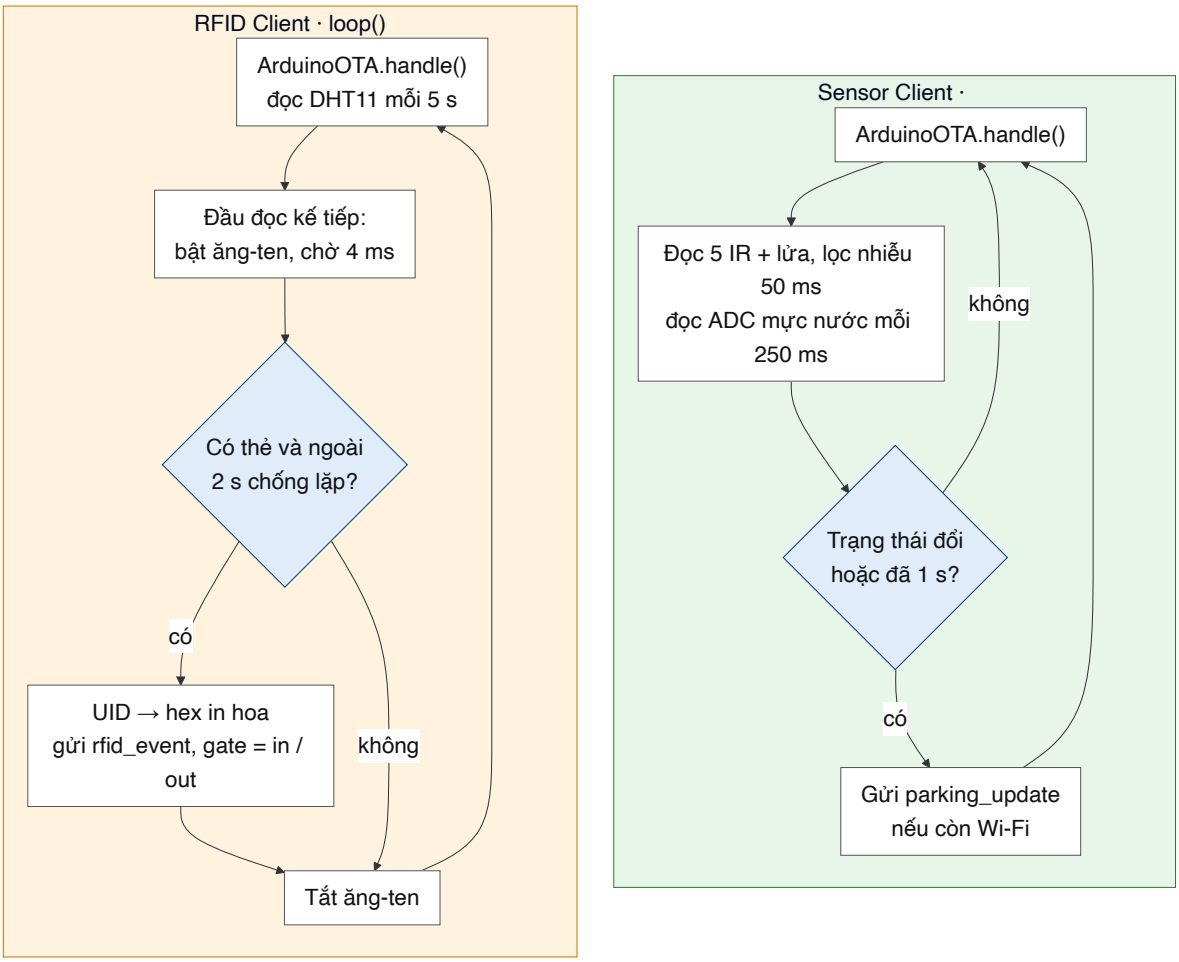
5.4.2 Xử lý bản tin UDP trên máy chủ



Hình 5.6: Lưu đồ điều phối bản tin trong tác vụ `udp_server`

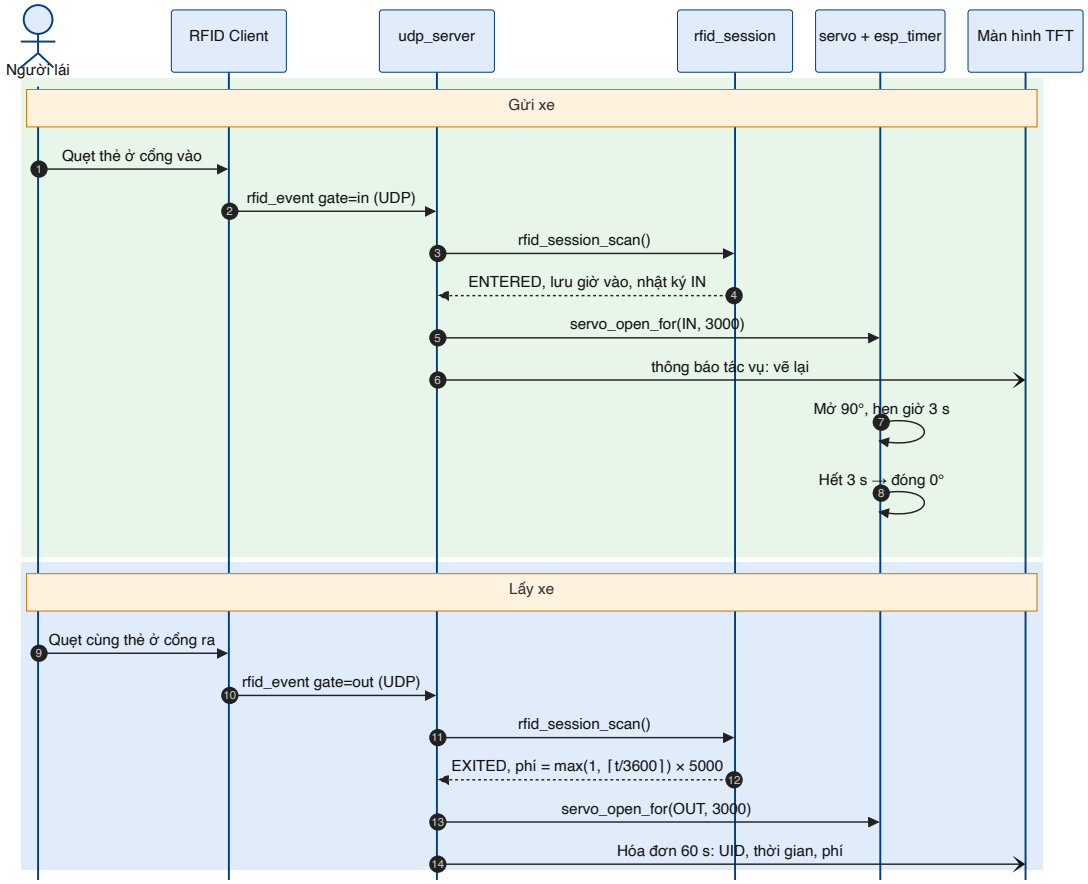
Nếu socket lỗi, tác vụ đóng socket, chờ 1 s rồi tạo lại — tự phục hồi mà không cần khởi động lại máy chủ. Khi không có gói, tác vụ ngủ trong `recvfrom()` và **không tốn CPU**.

5.4.3 Vòng lặp của hai client



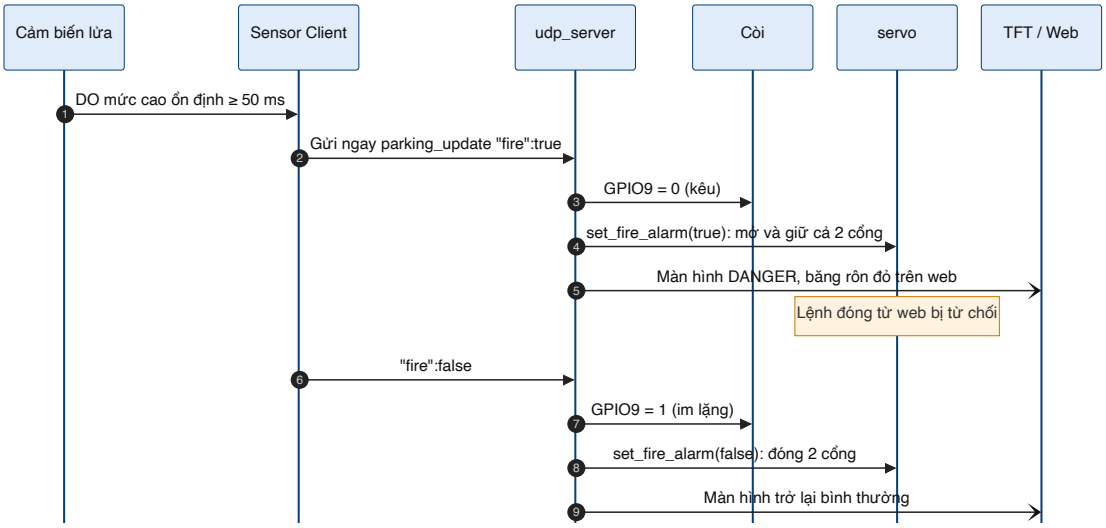
Hình 5.7: Vòng lặp không chặn của Sensor Client và RFID Client

5.4.4 Xe vào và ra bằng thẻ RFID



Hình 5.8: Biểu đồ tuần tự gửi xe (trên) và lấy xe (dưới)

5.4.5 Phát hiện cháy

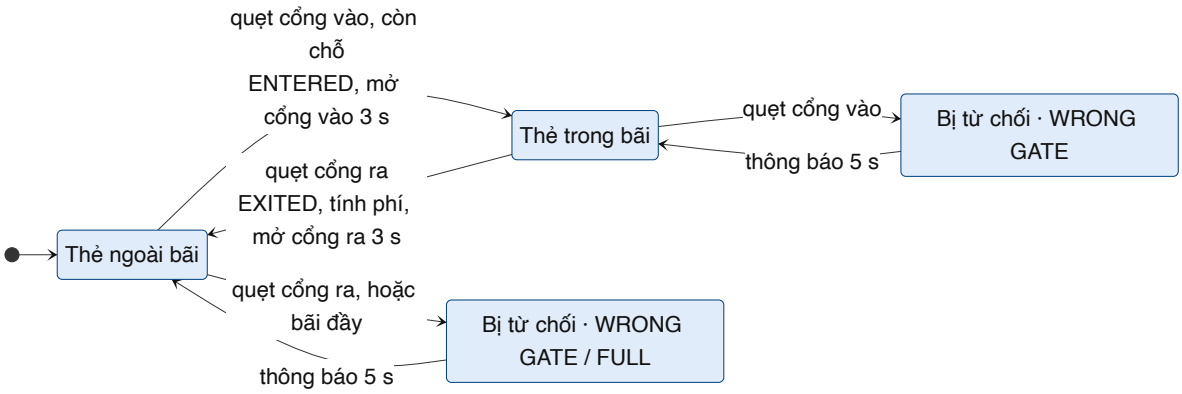


Hình 5.9: Biểu đồ tuần tự cảnh báo cháy

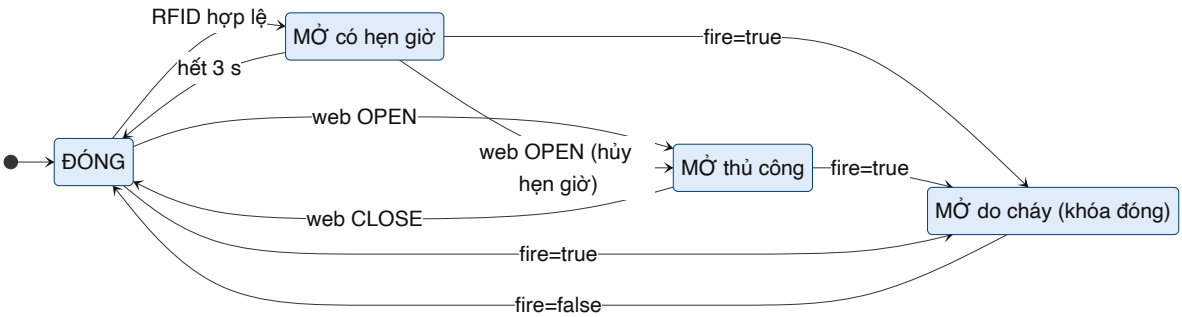
5.4.6 Trang web giám sát

Trình duyệt tải trang một lần, sau đó gọi `GET /api/parking` mỗi 1 s, `GET /api/rfid` mỗi 2 s và `GET /api/barrier` khi mở trang. Lệnh điều khiển dùng `POST`; các nút bị khóa đến khi có phản hồi để tránh gửi trùng.

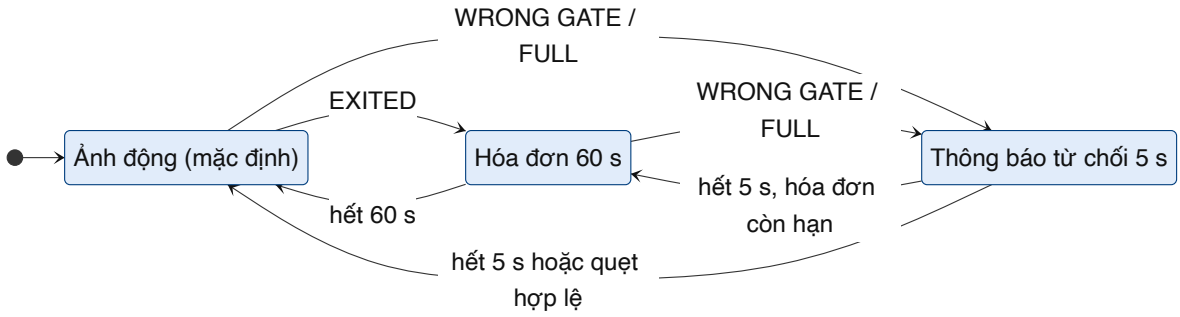
5.5 Máy trạng thái



Hình 5.10: Máy trạng thái của một thẻ RFID



Hình 5.11: Máy trạng thái của một thanh chắn



Hình 5.12: Trạng thái vùng chân màn hình (footer) TFT; khi có cháy, toàn màn hình chuyển sang màn hình cảnh báo

5.6 Giao thức và giao diện lập trình

```
{"type":"parking_update","slots":[1,0,0,1,0],"water_mm":0.2,"fire":false}
```

Mã nguồn 5.1: Bản tin `parking_update` (Sensor Client → máy chủ)

```
{"type":"rfid_event","card_id":"A1B2C3D4","gate":"in","temperature_c":
    28.0}
```

Mã nguồn 5.2: Bản tin `rfid_event` (RFID Client → máy chủ)

Bảng 5.4: Đặc tả trường dữ liệu và ràng buộc kiểm tra trên máy chủ

Trường	Kiểu	Ràng buộc
`type`	chuỗi	`"parking_update"` hoặc `"rfid_event"`
`slots`	mảng	Đúng 5 phần tử, mỗi phần tử 0 (trống) hoặc 1 (có xe)
`water_mm`	số	$0 \le x \le 10\,000$
`fire`	boolean	Tùy chọn (client cũ có thể không gửi); không chấp nhận 0/1
`card_id`	chuỗi	1–31 ký tự `[A-Za-z0-9_-]`, chuẩn hóa thành chữ hoa
`gate`	chuỗi	`"in"` hoặc `"out"` (không phân biệt hoa thường)
`temperature_c`	số	$-40 \le x \le 125$
Chung	—	Khóa lạ, khóa lặp, dữ liệu thừa sau `}`, chuỗi có ký tự thoát đều bị từ chối

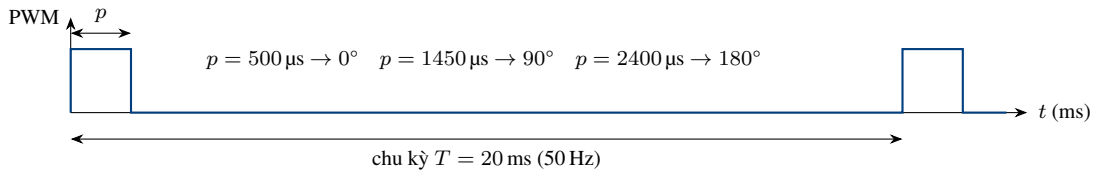
Bảng 5.5: API HTTP của máy chủ (11 route)

PT	Đường dẫn	Kết quả
GET	`/`, `/style.css`	Trang web nhúng trong firmware
GET	`/api/parking`	`received`, `slots`, `free`, `total`, `temperature_c`, `water_mm`, `fire`
GET	`/api/barrier`	`{"in_open":bool,"out_open":bool}`
GET	`/api/rfid`	`now_s`, `fee_per_hour`, `parked[]` (thẻ, giờ vào, phí tạm tính), `log[]` (10 sự kiện mới nhất)
POST	`/api/barrier/<cổng>/<lệnh>`	Cổng `in`, `out`, `all`; lệnh `open`, `close`. Lỗi 500 nếu bị từ chối (ví dụ đóng khi có cháy)

5.7 Thiết kế chi tiết từng thành phần

5.7.1 Thanh chắn: PWM phần cứng và hẹn giờ một lần

Servo SG90 nhận xung chu kỳ 20 ms; độ rộng xung quyết định góc (Hình 5.13).



Hình 5.13: Dạng xung điều khiển servo SG90

$$p(\theta) = 500 + \frac{1900\,\theta}{180}\ \mu\text{s}, \qquad D = \left\lfloor \frac{p \cdot (2^{14} - 1)}{20\,000} \right\rfloor \tag{5.1}$$

Với $\theta = 90°$: $p = 1450\,\mu$s, $D = 1187$; với $\theta = 0°$: $p = 500\,\mu$s, $D = 409$.

Tại sao 50 Hz, 14 bit, LEDC. **50 Hz** là tần số chuẩn của servo RC analog. **14 bit** là độ phân giải lớn nhất của LEDC trên ESP32-C3: mỗi bước $20\,000/16\,384 \approx 1{,}22\,\mu\text{s} \approx 0{,}12°$, mịn hơn nhiều vùng chết $\approx 1°$ của SG90 nên servo không rung. **LEDC** là PWM phần cứng: xung phát liên tục mà CPU không can thiệp, servo giữ góc ổn định dù Wi-Fi hay màn hình đang bận. Dải 500–2400 µs là giá trị đo thực tế của SG90 (danh nghĩa 1000–2000 µs không đủ 180°).

Tại sao dùng hẹn giờ một lần (`esp_timer`) thay vì tác vụ hỏi vòng. Phiên bản đầu dùng một tác vụ thức dậy 20 lần/giây để kiểm tra hạn đóng. Phiên bản hiện tại tạo một `esp_timer` một lần cho mỗi cổng: hàm đóng được gọi *đúng* lúc hết 3 s (sai số micro giây thay vì 50 ms), CPU không bị đánh thức vô ích và tiết kiệm 3 KB stack. Vì một lệnh khác có thể

> "đua" với hàm đóng, hàm đóng kiểm tra lại hạn đóng dưới mutex; lệnh tay từ web đặt hạn về 0 và dừng hẹn giờ — **lệnh của người vận hành luôn thắng**.

Hai cổng được mô tả bằng cùng một cấu trúc (kênh LEDC, chân, hướng lắp, trạng thái, hạn đóng, hẹn giờ) nên một hàm xử lý cho cả hai. Thanh chắn ra lắp đối xứng nên góc mở/đóng đảo ngược (0° mở, 90° đóng) — chỉ khai báo ở một hàm. Cả hai cổng **luôn khởi động ở vị trí đóng**, nên trạng thái không cần lưu vào flash; phiên bản trước ghi NVS mỗi lần thanh chắn chuyển động đã được bỏ (chậm và làm mòn flash vô ích).

5.7.2 Màn hình: SPI DMA, vẽ theo sự kiện, ảnh động tiền xử lý

> **Tại sao SPI 40 MHz và DMA**. Một khung hình đầy đủ là $240 \times 240 \times 16 = 921\,600$ bit; ở 40 MHz mất ≈ 23 ms, tối đa ≈ 43 khung/giây — dư cho giao diện tĩnh và màn hình cháy 8 khung/giây. 40 MHz bằng đúng $80\,\text{MHz}/2$ (không sai số chia tần) và dưới giới hạn ghi $\approx$ 62 MHz của ST7789, chừa biên an toàn cho dây cắm. DMA cho phép CPU làm việc khác trong lúc ảnh được đẩy ra.

- **Framebuffer RGB565** 115 200 byte: toàn bộ giao diện được vẽ vào RAM rồi gửi một lần. Màu được lưu sẵn theo thứ tự byte của ST7789 (byte cao trước) vì ESP32-C3 là little-endian.
- **Hàm tô hình chữ nhật** cắt biên một lần và tô từng dòng liên tục thay vì kiểm tra biên và đảo byte cho từng điểm ảnh.
- **Font 5×7 tự định nghĩa** cho A–Z, 0–9 (180 byte) — không cần thư viện đồ họa lớn như LVGL cho một giao diện cố định.
- **Vẽ theo sự kiện**: `udp_server` đánh thức tác vụ màn hình bằng thông báo tác vụ ngay khi có dữ liệu; nếu không có gì, tác vụ chỉ thức mỗi 1 s (để hết hạn hóa đơn/thông báo) hoặc 120 ms khi có cháy (để lửa nhấp nháy). Màn hình chỉ vẽ lại khi nội dung thực sự thay đổi.

Bảng 5.6: Dữ liệu ảnh động sau tiền xử lý và nén LZSS

Ảnh	Kích thước	Số khung	Dữ liệu thô	Sau nén
Kaoruko	96×96	10	184 320 B	70 564 B (2,6×)
Claude	68×46	18	112 608 B	16 204 B (6,9×)

> **Tại sao tiền xử lý ảnh trên máy tính**. Ảnh gốc có nền trắng cần xóa và điểm ảnh cần đảo byte. Phiên bản trước làm việc này trên chip cho *mọi khung hình*, 10 lần/giây: thuật toán loang tìm nền trắng nối với viền (cần thêm 1,2 KB RAM) rồi đảo byte 9 216 điểm. Vì ảnh không bao giờ đổi, nhóm làm một lần trên máy tính và lưu kết quả đã xử lý: firmware chỉ còn giải nén. Kết quả được kiểm chứng giống từng điểm ảnh với cách cũ trên cả 28 khung,

dữ liệu còn nhỏ hơn (86 768 so với 87 977 byte). LZSS có bộ giải nén ≈ 40 dòng, không cần bộ nhớ phụ và kiểm tra biên đầy đủ nên dữ liệu hỏng không gây ghi tràn.

Hai bộ đệm DMA ($2 \times 18\,432$ byte) được dùng luân phiên cho ảnh động: lệnh vẽ chỉ *xếp hàng* giao dịch DMA rồi trả về, nên giải nén khung kế tiếp vào cùng bộ đệm khi DMA chưa đọc xong sẽ làm hình bị xé. Hai tác vụ cùng vẽ lên một panel nên dùng chung một mutex.

5.7.3 Bộ kiểm tra bản tin JSON

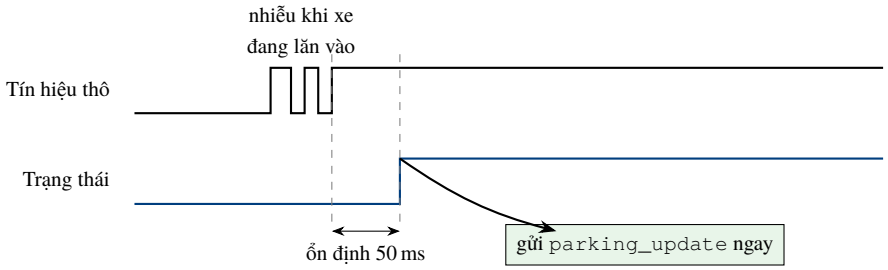
Tại sao tự viết thay vì dùng thư viện cJSON. Gói UDP là *ranh giới tin cậy*: bất kỳ thiết bị nào trong mạng đều gửi được. cJSON cấp phát heap cho từng nút (phân mảnh heap khi chạy lâu) và chấp nhận mọi JSON hợp lệ. Bộ đọc tự viết (`json_scan.c`, 81 dòng, dùng chung cho cả hai loại bản tin) **không cấp phát động**, chỉ chấp nhận đúng lược đồ (Bảng 5.4) và có **vòng lặp giới hạn** (tối đa 4 khóa) nên thời gian xử lý bị chặn trên. Bộ đệm nhận 256 byte, đọc tối đa 255 byte và luôn kết thúc bằng `'\0'`.

5.7.4 Phiên gửi xe, phí, thông báo và nhật ký

- **Phiên**: mảng tĩnh 5 phần tử — bằng số chỗ đỗ, không có nhiều xe hơn số chỗ. Thời điểm vào lấy từ `esp_timer_get_time()` (64 bit, micro giây, không tràn trong hơn 290 000 năm).
- **Phí**: $h = \max(1, \lceil t/3600 \rceil)$, phí $= 5000\,h$ VNĐ, tính bằng số nguyên `(t + 3599) / 3600`, bão hòa ở `UINT32_MAX` nếu tràn.
- **Thông báo từ chối** (sai cổng, bãi đầy): giữ 5 s; lần quẹt hợp lệ tiếp theo xóa thông báo ngay.
- **Nhật ký**: bộ đệm vòng 10 sự kiện (≈ 520 byte), đọc ra theo thứ tự mới nhất trước; không dùng heap.

Tại sao chọn các thông số này. **5.000 VNĐ/giờ, làm tròn lên, tối thiểu 1 giờ** là cách tính phổ biến của bãi xe, dễ giải thích. **Hóa đơn 60 s** đủ để tài xế đọc và trả tiền; **thông báo 5 s** đủ đọc mà không che hóa đơn quá lâu. **Thanh chắn mở 3 s** đủ cho xe mô hình đi qua nhưng không đủ để xe sau "đi ké". **Nhật ký 10 sự kiện** vừa một màn hình điện thoại và tốn dưới 1 KB RAM.

5.7.5 Sensor Client: lọc nhiễu và gửi theo thay đổi



Hình 5.14: Chống rung: chỉ chấp nhận mức mới khi giữ ổn định 50 ms, rồi gửi ngay

Tại sao lọc 50 ms và gửi theo thay đổi kèm nhắc lại 1 s. Mép xe và bánh xe làm tín hiệu IR bật/tắt liên tục trong vài chục mili giây khi xe đang vào chỗ. Nhóm tinh chỉnh thực nghiệm từ 250 ms xuống **50 ms**: đủ lọc nhiễu mà cảm giác tức thời. Trước đây trạng thái chỉ được gửi theo chu kỳ 1 s nên thay đổi có thể trễ đến 1 s; nay **gửi ngay khi có thay đổi**, đồng thời vẫn **nhắc lại mỗi giây** để một gói bị mất được sửa ở lần sau. Mỗi cảm biến có bộ đếm riêng.

Mực nước được quy đổi tuyến tính từ ADC 10 bit:

$$h_{\text{mm}} = \begin{cases} 0, & raw \le 20 \\ \dfrac{\min(raw, 700) - 20}{700 - 20} \cdot 40, & raw > 20 \end{cases} \tag{5.2}$$

trong đó 20 là mức nhiễu khi khô, 700 là giá trị khi ngập hoàn toàn và 40 mm là chiều dài vùng đo; ADC được đọc mỗi 250 ms vì đọc quá dày ảnh hưởng Wi-Fi trên ESP8266 trong khi mực nước thay đổi chậm.

5.7.6 RFID Client: hai đầu đọc trên một bus SPI

Tại sao bật ăng-ten luân phiên, chờ 4 ms, gain 48 dB, chống lặp 2 s. Hai đầu đọc 13,56 MHz đặt gần nhau sẽ **nhiễu chéo**; chỉ bật một ăng-ten tại một thời điểm loại bỏ hoàn toàn vấn đề. Thẻ thụ động cần vài mili giây để nhận đủ năng lượng: **4 ms** đủ cho thẻ MIFARE mà chu kỳ quét vẫn ngắn. **Gain 48 dB** bù tầm đọc khi đầu đọc nằm sau vỏ mô hình. Thẻ để yên sẽ bị đọc liên tục; **chống lặp 2 s** áp dụng cho cả hai đầu đọc, tránh một lần quẹt bị gửi nhiều lần làm "vào" biến thành "ra". UID được đổi sang chuỗi hex bằng **bộ đệm cố định** thay vì `String` của Arduino: không cấp phát heap mỗi lần quẹt, nên heap 80 KB của ESP8266 không bị phân mảnh khi chạy lâu. DHT11 đọc mỗi **5 s** vì cảm biến cần ít nhất 1 s giữa hai lần đọc và nhiệt độ thay đổi chậm.

5.7.7 Trang web

Tại sao hỏi vòng 1–2 s thay vì WebSocket. Hỏi vòng không giữ trạng thái kết nối trên máy chủ: mỗi request độc lập, xử lý xong giải phóng ngay — phù hợp giới hạn socket và RAM, và đơn giản khi nhiều trình duyệt cùng xem. Với dữ liệu thay đổi theo giây, độ trễ thêm tối đa 1 s là chấp nhận được. Trang web là HTML/CSS/JavaScript thuần, nhúng trong firmware bằng `EMBED_TXTFILES`: không cần phân vùng tệp, không phụ thuộc CDN (điện thoại kết nối hotspot không có Internet).

5.8 Phân tích kiến trúc

5.8.1 Hiệu năng và độ trễ

Bảng 5.7: Ngân sách độ trễ của các luồng chính (ước tính từ thông số thiết kế)

Luồng	Thành phần	Tổng
Xe vào ô → màn hình TFT	lọc 50 ms + gửi ngay + UDP vài ms + đánh thức + vẽ 23 ms	$\approx 0{,}1$ s
Xe vào ô → trang web	như trên + hỏi vòng tối đa 1 s	$\leq 1{,}1$ s
Quẹt thẻ → thanh chắn mở	quét 2 đầu đọc $\approx$ 10 ms + UDP + xử lý $<$ 1 ms + servo 90° $\approx 0{,}15$ s	$\approx 0{,}2$ s
Có lửa → còi kêu	lọc 50 ms + gửi ngay + UDP	$\approx 0{,}06$ s
Mất một gói `parking_update`	sửa ở lần nhắc lại	≤ 1 s

5.8.2 Bộ nhớ

Bảng 5.8: Sử dụng bộ nhớ của firmware máy chủ (đo bằng `idf.py size`)

Hạng mục	Kích thước
DRAM tĩnh dùng / tổng (mã IRAM 81 444 + `.data` 13 792 + `.bss` 21 504)	116 740 / 321 296 B (36,3%)
Framebuffer màn hình (heap, DMA)	115 200 B
2 bộ đệm ảnh động (heap, DMA)	36 864 B
Stack các tác vụ của ứng dụng	$\approx$ 19 KB
Bảng phiên, hóa đơn, thông báo, nhật ký	$<$ 1 KB
Ảnh firmware	1 083 536 B
Phân vùng ứng dụng (`factory`)	1 536 000 B (dùng 70,5%)

Tại sao phân vùng ứng dụng 1,5 MB. Bảng phân vùng mặc định chỉ có 1 MB cho ứng dụng và đã đầy 98% trước khi thêm mDNS. Flash 2 MB đủ chỗ cho phân vùng 1,5 MB có sẵn trong ESP-IDF (`partitions_singleapp_large.csv`), được khai báo trong `sdkconfig.defaults` để mọi thành viên build ra cùng cấu hình; còn 29% dự phòng cho tính năng mới.

5.8.3 Độ tin cậy và khả năng chịu lỗi

Bảng 5.9: Phân tích lỗi và cách hệ thống phản ứng

Tình huống lỗi	Phản ứng của hệ thống
Mất gói UDP cảm biến	Bản tin nhắc lại sau ≤ 1 s sửa trạng thái
Bản tin sai định dạng / độc hại	Bị từ chối, trạng thái cũ giữ nguyên, có cảnh báo trong log
Socket UDP lỗi	Đóng socket, chờ 1 s, tạo lại
Máy chủ mất Wi-Fi	Tự kết nối lại; nhận IP (có thể mới) → thông báo lại `parking.local`
Client mất Wi-Fi	Sensor Client bỏ qua lần gửi, tự kết nối lại; trang web báo OFFLINE
DHT11 đọc lỗi	Giữ giá trị hợp lệ gần nhất
Có cháy khi người vận hành bấm đóng	Lệnh đóng bị từ chối; thanh chắn giữ mở
Máy chủ khởi động lại	Thanh chắn về vị trí đóng; phiên và nhật ký (RAM) được làm mới
Lỗi khởi tạo phần cứng	Fail-fast: in lỗi và khởi động lại

5.8.4 Bảo mật

- **Đã có**: kiểm tra bản tin chặt chẽ ở ranh giới mạng; không cấp phát động khi xử lý dữ liệu ngoài; mã thẻ giới hạn ký tự nên không thể chèn mã vào JSON hay HTML; trang web chèn dữ liệu bằng `textContent`, không dùng `innerHTML` cho dữ liệu từ máy chủ; mạng dùng WPA2.
- **Còn hạn chế**: thiết bị nào biết mật khẩu Wi-Fi đều gửi được bản tin giả (giả thẻ, báo cháy giả) và gọi API mở thanh chắn; mật khẩu cố định trong mã nguồn. Hướng khắc phục: mã xác thực HMAC cho bản tin và đăng nhập cho API (Chương 8).

5.8.5 Khả năng bảo trì, kiểm thử và mở rộng

- Lớp nghiệp vụ độc lập phần cứng: 250 phép kiểm thử chạy trên máy tính trong vài giây.
- Hai client được kiểm thử bằng chính mã nguồn `.ino` trên phần cứng giả lập, kèm kiểm tra hợp đồng giao tiếp với bộ đọc JSON thật của máy chủ.
- Số chỗ đỗ là hằng số `PARKING_SLOT_COUNT`; số phiên tối đa bằng số chỗ. Mở rộng quy mô lớn hơn cần đổi định dạng `slots` và tách cảm biến theo khu vực.

5.8.6 Tổng hợp quyết định kiến trúc

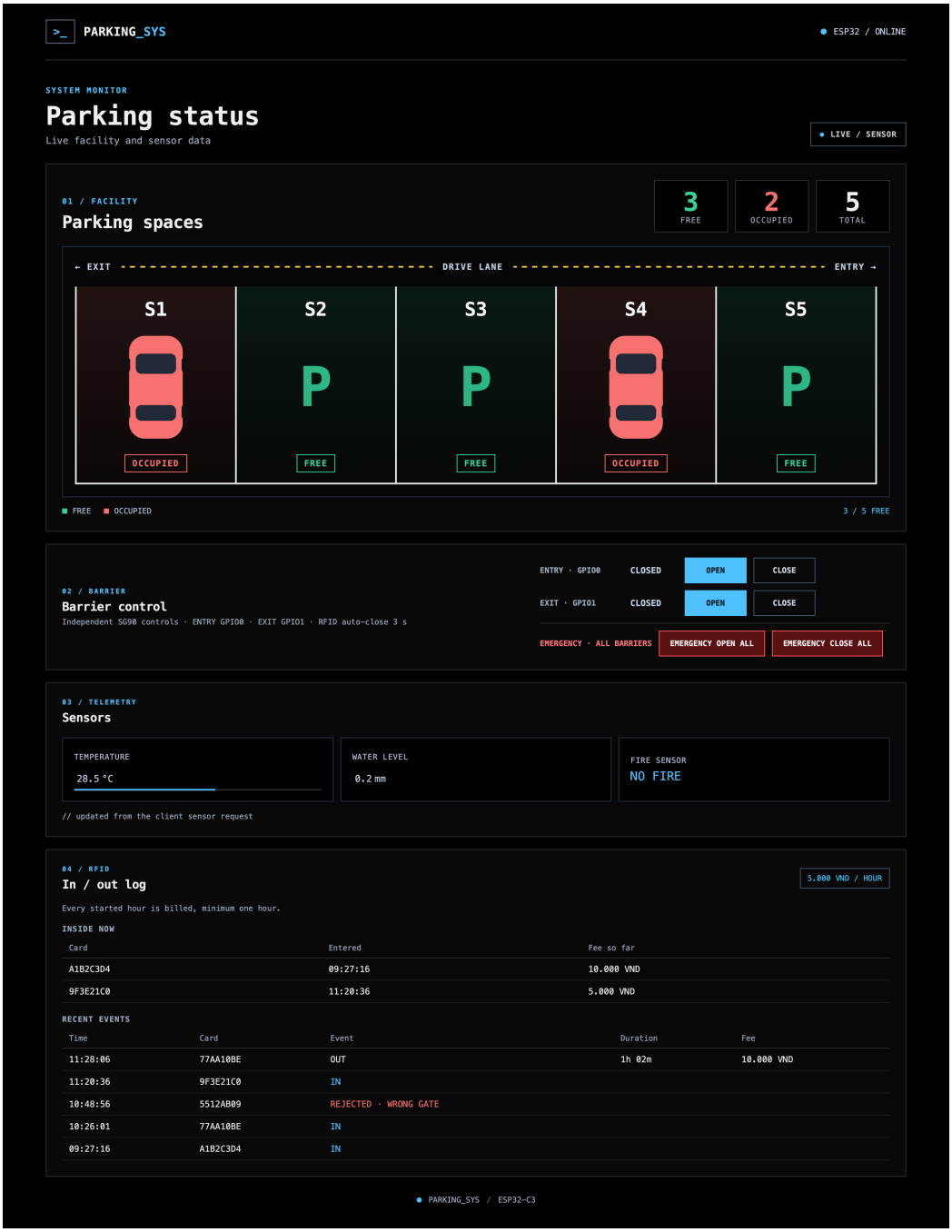
Bảng 5.10: Các quyết định kiến trúc chính

Quyết định	Phương án khác	Lý do / đánh đổi
3 bo mạch, máy chủ trung tâm	1 bo lớn	Đủ chân, đặt cảm biến gần nơi cần đo, cô lập lỗi; đổi lại cần mạng
FreeRTOS trên máy chủ	Vòng lặp Arduino	Nhiều việc song song có ưu tiên; đổi lại phải đồng bộ cẩn thận
UDP broadcast + JSON	TCP, MQTT, nhị phân	Nhẹ, dễ gỡ lỗi; mất gói được bù bằng nhắc lại
Gửi theo thay đổi + nhắc lại 1 s	Chỉ gửi định kỳ	Độ trễ $\approx$ 0,1 s; vẫn tự sửa khi mất gói
Điểm truy cập trên RFID Client + mDNS	AP trên máy chủ, router	Dành RAM máy chủ cho màn hình; tên cố định dù IP đổi
Bộ đọc JSON tự viết	cJSON	Không cấp phát, chỉ đúng lược đồ; đổi lại phải tự kiểm thử kỹ
Hẹn giờ một lần cho thanh chắn	Tác vụ hỏi vòng	Chính xác, không đánh thức CPU vô ích
Vẽ màn hình theo sự kiện	Hỏi vòng 500 ms	Phản hồi ngay, ít lần thức
Tiền xử lý ảnh động trên máy tính	Xử lý trên chip mỗi khung	Bớt CPU và 1,2 KB RAM; dữ liệu nhỏ hơn
Phiên/nhật ký trong RAM	Lưu flash	Đơn giản, không mòn flash; mất khi khởi động lại

5.9 Demo

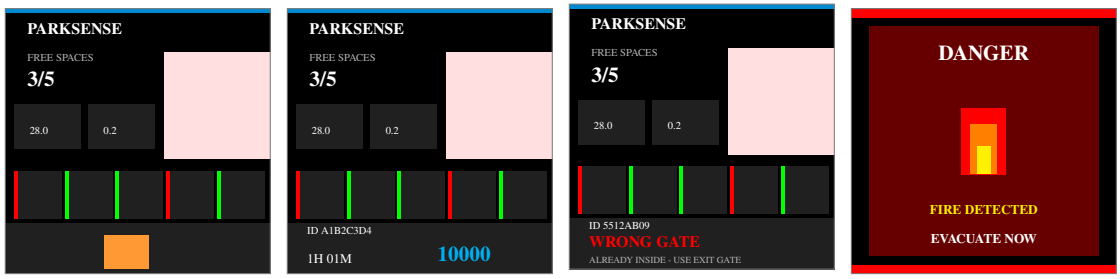
5.9.1 Trang web giám sát

Hình 5.15 là ảnh chụp trang web thật của hệ thống (chạy với dữ liệu mẫu). Sơ đồ bãi là phần chính: 5 ô lớn, ô có xe vẽ hình xe màu đỏ, ô trống hiện chữ P màu xanh, làn xe với hướng vào/ra; phía trên là số chỗ trống/có xe/tổng. Bên dưới là điều khiển thanh chắn, cảm biến và nhật ký vào/ra kèm giá gửi xe.



Hình 5.15: Trang web giám sát tại `http://parking.local` (dữ liệu mẫu)

5.9.2 Các màn hình TFT



(a) Bình thường (b) Hóa đơn khi ra (c) Quẹt sai cổng (d) Có cháy

Hình 5.16: Bốn trạng thái màn hình TFT 240×240 (mô phỏng bố cục theo mã nguồn `lcd_display.c`)

Trên màn hình bình thường: hàng trên là số chỗ trống, hai ô nhiệt độ và mực nước, ảnh động 96×96 ở góc phải, hàng giữa là 5 ô S1–S5 với vạch xanh (trống)/đỏ (có xe), vùng chân dùng cho ảnh động nhỏ, hóa đơn hoặc thông báo.

5.9.3 Kịch bản demo

1. Bật RFID Client, máy chủ, Sensor Client; mở `http://parking.local` trên điện thoại.
2. Đặt xe mô hình vào ô S2: ô S2 trên TFT chuyển đỏ ngay, trên web hiện hình xe trong $\leq$ 1 s.
3. Quẹt thẻ ở cổng vào: thanh chắn vào mở rồi tự đóng sau 3 s; web có dòng `IN`.
4. Quẹt lại thẻ đó ở cổng vào: màn hình báo `WRONG GATE – ALREADY INSIDE – USE EXIT GATE`.
5. Quẹt thẻ ở cổng ra: hóa đơn hiện trên TFT, thanh chắn ra mở; web có dòng `OUT` kèm phí.
6. Đưa bật lửa gần cảm biến lửa: còi kêu, màn hình DANGER, hai thanh chắn mở và không đóng được từ web.
7. Tắt lửa: còi tắt, thanh chắn đóng, màn hình trở lại bình thường.

Chương 6. Hướng dẫn chạy mã nguồn từng bước

Chương này hướng dẫn từ lúc lấy mã nguồn đến lúc hệ thống chạy hoàn chỉnh, kèm kết quả mong đợi ở mỗi bước để kiểm tra. Môi trường cần cài trước theo Chương 4. Có hai cách chạy:

- **Đầy đủ phần cứng**: Bước 1 đến Bước 9.
- **Không cần client** (chỉ có bo máy chủ, dùng máy tính giả lập hai client): Bước 1, 2, 4 rồi Bước 10.

Bộ kiểm thử tự động (không cần phần cứng nào) ở Bước 11.

Bảng 6.1: Tên cổng nối tiếp theo hệ điều hành

Hệ điều hành	ESP32-C3 SuperMini (USB tích hợp)	ESP8266 NodeMCU (CH340/CP2102)
Windows	`COM3`, `COM4`, … (xem Device Manager)	`COM5`, …
macOS	`/dev/cu.usbmodem*`	`/dev/cu.usbserial-*` hoặc `/dev/cu.wchusbserial*`
Linux	`/dev/ttyACM0`	`/dev/ttyUSB0`

Bước 1. Lấy mã nguồn

```
git clone https://github.com/thinhdinh25/INT3108_embedded-Smart-Parking-
    Project.git
cd INT3108_embedded-Smart-Parking-Project
```

Hoặc tải tệp ZIP từ trang GitHub của dự án và giải nén.

Bước 2. Làm quen cấu trúc thư mục

```
  Server/              Firmware may chu ESP32-C3 (ESP-IDF)
    main/              Ma nguon: barrier, buzzer, display, network, parking
    sdkconfig.defaults Cau hinh mac dinh: -O2, phan vung ung dung 1,5 MB
  SensorClient/        Sketch Arduino cho Sensor Client (ESP8266)
  RFIDClient/          Sketch Arduino cho RFID Client (ESP8266)
  examples/            Cong cu gia lap client bang Python
  tests/               Bo kiem thu tu dong tren may tinh
  docs/                Ghi chu dau day
  report/              Bao cao nay (LaTeX + PDF)
```

Mã nguồn 6.1: Cấu trúc kho mã nguồn

Bước 3. Lắp mạch

Đấu dây theo Bảng 3.7 (máy chủ) và Bảng 3.8 (hai client). Kiểm tra trước khi cấp nguồn:

- Servo dùng nguồn 5 V riêng, **GND nối chung** với ESP32-C3; không cấp servo từ chân 3V3.

- Màn hình ST7789 dùng 3V3; RC522 dùng 3V3 (không dùng 5 V).
- Module còi không được kéo GPIO9 xuống mức thấp khi cấp nguồn (nếu không, bo sẽ vào chế độ nạp firmware).

Bước 4. Biên dịch và nạp máy chủ ESP32-C3

Mở terminal ESP-IDF (Chương 4), cắm ESP32-C3 qua USB:

```
cd Server
idf.py set-target esp32c3        # chi lan dau; tao sdkconfig tu sdkconfig.
    defaults
idf.py build                     # lan dau tai them thanh phan espressif/
    mdns
idf.py -p <CONG> flash monitor   # vi du: -p COM4 hoac -p /dev/cu.
    usbmodem101
```

Kết quả mong đợi sau `idf.py build`:

```
  INT3108_Smart_Parking_Project.bin binary size 0x108890 bytes. Smallest
      app
  partition is 0x177000 bytes. 0x6e770 bytes (29%) free.
  Project build complete.
```

Kết quả mong đợi trên cửa sổ monitor (thoát bằng `Ctrl+]`):

```
  I (...) LCD_DISPLAY: ST7789 display started at 240x240 with 10-frame
      Kaoruko animation
  I (...) WIFI_STA: Wi-Fi station started, connecting to SSID=ESP32-Parking
      ...
  I (...) HTTP_SERVER: HTTP server listening on port 80
  I (...) UDP_SERVER: UDP server listening on port 3333
  I (...) SERVER: Web interface: http://parking.local
```

Màn hình TFT hiện giao diện `PARKSENSE` với các giá trị `--.-`; hai thanh chắn về vị trí đóng. Máy chủ sẽ báo `Got IP` khi RFID Client (điểm truy cập) đã bật ở Bước 5.

> **Lưu ý khi đã có cấu hình cũ.** `sdkconfig.defaults` chỉ được áp dụng khi chưa có tệp `Server/sdkconfig`. Nếu máy đã từng build bản cũ, hãy chạy lại `idf.py set-target esp32c3` (tạo lại cấu hình) hoặc chọn trong `idf.py menuconfig`: *Compiler options* → *Optimize for performance* và *Partition Table* → *Single factory app (large), no OTA*. Lần nạp đầu sau khi đổi bảng phân vùng phải dùng `idf.py flash` (nạp cả bảng phân vùng).

Bước 5. Nạp RFID Client (điểm truy cập Wi-Fi)

1. Mở Arduino IDE, *File* → *Open* tệp `RFIDClient/RFIDClient.ino`.
2. *Tools* → *Board*: **NodeMCU 1.0 (ESP-12E Module)**; *Port*: cổng của bo (Bảng 6.1); *Upload Speed*: 115200.

3. Bấm **Upload** (mũi tên). Chờ `Done uploading`.
4. Mở *Serial Monitor*, tốc độ **115200**.

Kết quả mong đợi: dòng `Đã phát WiFi Hotspot!`, `SSID: ESP32-Parking`, `Địa chỉ IP của Hotspot: 192.168.4.1`, phiên bản firmware của hai đầu đọc và `Hệ thống đã sẵn sàng!`. Trên monitor của máy chủ xuất hiện:

```
I (...) WIFI_STA: Got IP: 192.168.4.2
I (...) SERVER: mDNS: parking.local -> 192.168.4.2
```

Bước 6. Nạp Sensor Client

Làm như Bước 5 với tệp `SensorClient/SensorClient.ino`. **Kết quả mong đợi** trên Serial Monitor: `Connected, client IP: 192.168.4.x`, sau đó mỗi giây một dòng `Sent: {"type":"parking_update",...}`. Màn hình TFT hiện số chỗ trống thật.

> **Nạp lại qua Wi-Fi (OTA).** Từ lần thứ hai, khi máy tính kết nối Wi-Fi `ESP32-Parking`, hai client xuất hiện trong *Tools* → *Port* dưới dạng cổng mạng (ví dụ `ESP8266-RC522-DHT11`); chọn cổng đó để nạp firmware mà không cần tháo bo khỏi mô hình.

Bước 7. Bật hệ thống theo thứ tự

Thứ tự khuyến nghị: **RFID Client** (phát Wi-Fi) → **máy chủ** → **Sensor Client**. Thứ tự khác vẫn chạy được vì máy chủ và Sensor Client tự kết nối lại, chỉ chậm hơn vài giây.

Bước 8. Mở trang web giám sát

1. Trên điện thoại/máy tính, kết nối Wi-Fi `ESP32-Parking`, mật khẩu `12345678`.
2. Mở trình duyệt, vào `http://parking.local`.
3. Nếu thiết bị không mở được tên `.local` (một số máy Android cũ), dùng địa chỉ IP trong dòng `Web interface: http://192.168.4.x` trên monitor của máy chủ.

Kết quả mong đợi: trang như Hình 5.15, góc trên phải hiện `ESP32 / ONLINE` và nhãn `LIVE / SENSOR`.

Bước 9. Chạy các kịch bản demo

Thực hiện kịch bản ở mục 5.9 và đối chiếu với Bảng 7.2 (Chương 7).

Bước 10. Chạy khi không có hai client (giả lập bằng máy tính)

1. Tạo điểm phát Wi-Fi trên điện thoại với tên `ESP32-Parking`, mật khẩu `12345678` (máy chủ sẽ tự kết nối vào), *hoặc* dùng RFID Client nếu có.
2. Cho máy tính kết nối cùng Wi-Fi; mở monitor máy chủ để xem IP (`Got IP:` …).

3. Giả lập Sensor Client và RFID Client (thay `<IP>` bằng IP máy chủ; nếu dùng RFID Client làm điểm phát thì có thể bỏ `--host`):

```
python3 examples/udp_sender.py --host <IP> --slots 1,0,0,1,0
python3 examples/udp_rfid_sender.py A1B2C3D4 --gate in  --host <IP>
python3 examples/udp_rfid_sender.py A1B2C3D4 --gate in  --host <IP>  # quet
    lai: WRONG GATE
python3 examples/udp_rfid_sender.py A1B2C3D4 --gate out --host <IP>  # ra:
    hoa don
python3 examples/udp_sender.py --host <IP> --slots 1,0,0,1,0 --fire  # bao
    chay
python3 examples/udp_sender.py --host <IP> --slots 1,0,0,1,0         # het
    chay
```

Kết quả mong đợi: TFT và trang web thay đổi đúng như khi dùng client thật; thanh chắn mở/đóng; nhật ký trên web có các dòng `IN`, `REJECTED · WRONG GATE`, `OUT`. Trên Windows dùng `python` thay cho `python3`.

Bước 11. Chạy bộ kiểm thử tự động

Không cần ESP-IDF hay Arduino, chỉ cần trình biên dịch C/C++:

```
make -C tests
```

Kết quả mong đợi:

```
  json_parsers              57/57 checks passed
  rfid_session              74/74 checks passed
  gif_frames                89/89 checks passed
  sensor_client             15/15 checks passed
  rfid_client               15/15 checks passed
  ALL TESTS PASSED
```

6.1 Xử lý sự cố thường gặp

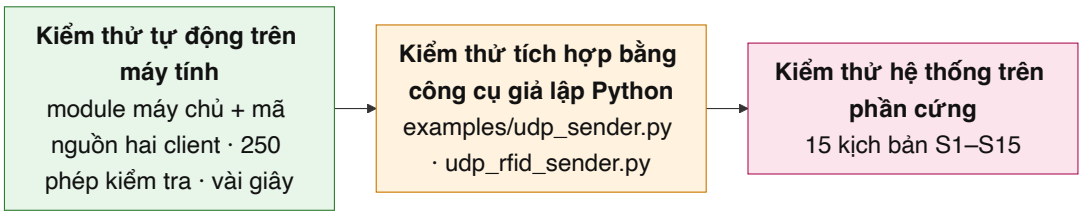
Bảng 6.2: Lỗi thường gặp và cách khắc phục

Hiện tượng	Nguyên nhân và cách khắc phục
`idf.py: command not found`	Chưa mở terminal ESP-IDF; mở shortcut/script kích hoạt do EIM tạo.
`Failed to connect to ESP32-C3`	Sai cổng; hoặc giữ nút BOOT, bấm RESET, thả BOOT rồi nạp lại.
Build báo phân vùng đầy	Đang dùng `sdkconfig` cũ (1 MB): làm theo hộp "Lưu ý khi đã có cấu hình cũ" ở Bước 4.

Hiện tượng	**Nguyên nhân và cách khắc phục**
Monitor lặp `Disconnected ... retrying`	RFID Client chưa bật hoặc sai SSID/mật khẩu; kiểm tra Bước 5.
Arduino không thấy cổng ESP8266	Cài driver CH340/CP2102; thử cáp USB khác (cáp chỉ sạc không truyền dữ liệu).
Thiếu `MFRC522.h` hoặc `DHT.h`	Cài thư viện như Chương 4.
`parking.local` không mở	Thiết bị không hỗ trợ mDNS: dùng IP trong log máy chủ; kiểm tra điện thoại đang ở đúng Wi-Fi `ESP32-Parking`.
Trang web báo `OFFLINE`	Máy chủ mất điện hoặc mất Wi-Fi; xem monitor máy chủ.
Servo rung, bo tự khởi động lại	Cấp servo từ 3V3 hoặc thiếu GND chung; dùng nguồn 5 V riêng.
Màn hình trắng hoặc lệch hình	Kiểm tra dây SPI; chỉnh chân/độ lệch trong `idf.py menuconfig → Parking TFT display`.
Ô đỗ báo ngược (trống thành có xe)	Chỉnh biến trở trên module IR; kiểm tra thứ tự chân S1–S5.

Chương 7. Kiểm thử

7.1 Chiến lược kiểm thử



Hình 7.1: Ba tầng kiểm thử: càng xuống dưới càng nhanh, tự động và chạy thường xuyên

7.2 Kiểm thử tự động

Bộ kiểm thử nằm trong `tests/` và chạy bằng `make -C tests`. Các module nghiệp vụ của máy chủ được biên dịch với hai header giả (critical section và đồng hồ điều khiển được). Hai sketch Arduino được biên dịch *nguyên vẹn* cùng lớp giả lập thư viện (`tests/arduino/`): đồng hồ `millis()`, mức logic từng chân, ADC, Wi-Fi, UDP, đầu đọc MFRC522 và DHT11.

Bảng 7.1: Các bộ kiểm thử tự động và kết quả

Bộ	Nội dung kiểm tra	Kết quả
`json_parsers`	Bản tin hợp lệ (mọi thứ tự khóa, khoảng trắng, giới hạn dải); 22 bản tin `parking_update` sai và 16 bản tin `rfid_event` sai đều bị từ chối (thiếu/thừa khóa, khóa lặp, sai kiểu, ngoài dải, ký tự lạ, dữ liệu thừa, chuỗi có ký tự thoát)	57/57
`rfid_session`	Phí theo giờ (biên 0, 3600, 3601 s, bão hòa); vào/ra, sai cổng; hóa đơn hết hạn đúng 60 s; bãi đầy, tái sử dụng chỗ; nhiệt độ; nhật ký vòng (thứ tự, chỉ giữ 10); thông báo từ chối 5 s	74/74
`gif_frames`	28 khung ảnh động giải nén đủ; nền đã xóa và byte đã đảo đúng; từ chối yêu cầu sai kích thước	89/89
`sensor_client`	Chạy `SensorClient.ino`: nhắc lại đúng 1 gói/giây; gửi ngay sau 50 ms khi có xe; lọc nhiễu 30 ms; thứ tự S1–S5; báo cháy và hết cháy; quy đổi mực nước; mất/có lại Wi-Fi; mọi gói được bộ đọc của máy chủ chấp nhận	15/15
`rfid_client`	Chạy `RFIDClient.ino`: UID hex in hoa; cổng vào/ra; chống lặp 2 s trên cả hai đầu đọc; UID 10 byte; DHT11 lỗi giữ giá trị cũ; mọi gói được bộ đọc của máy chủ chấp nhận	15/15
Tổng		**250/250**

```
/* A car on S1 is reported as soon as the 50 ms debounce passes. */
fake::pin_level[SLOT_PINS[0]] = LOW;
run_for(40);
CHECK(fake::udp_sent.empty());            /* still inside the debounce
    window */
run_for(30);
CHECK(fake::udp_sent.size() == 1);
CHECK(last_packet(&s) && s.slots[0] == 1); /* parsed by the real server
    parser */
```

Mã nguồn 7.1: Ví dụ kiểm thử chống rung của Sensor Client (`tests/test_sensor_client.cpp`)

> **Tại sao kiểm thử trên máy tính.** Lỗi trong bộ đọc bản tin, cách tính phí hay logic chống rung rất khó tái hiện trên phần cứng (phải đúng thời điểm, đúng chuỗi byte). Trên máy tính, đồng hồ giả cho phép kiểm tra chính xác đến từng mili giây, cả bộ chạy trong vài giây và chạy lại sau mỗi lần sửa mã. Việc đưa gói tin của client vào bộ đọc thật của máy chủ bảo đảm hai bên luôn khớp định dạng — lỗi kiểu "client gửi `temperature_c` trong `parking_update` bị máy chủ từ chối" được phát hiện ngay.

Bộ kiểm thử cũng đã giúp phát hiện một lỗi *trong chính bài kiểm thử*: khi kiểm tra "bãi đầy", một thẻ từ bài kiểm thử trước vẫn còn trong bãi khiến chỉ còn 4 chỗ trống. Bài kiểm thử được sửa để tự dọn trạng thái — đúng như mục đích của việc chạy kiểm thử thường xuyên.

7.3 Kiểm thử tích hợp bằng công cụ giả lập

Các lệnh ở Bước 10 (Chương 6) gửi bản tin thật tới máy chủ thật; dùng để kiểm tra màn hình, thanh chắn, còi, trang web và nhật ký khi chưa có đủ client.

7.4 Kịch bản kiểm thử hệ thống trên phần cứng

Bảng 7.2: Kịch bản kiểm thử trên phần cứng và hành vi mong đợi

#	Thao tác	Hành vi mong đợi
S1	Cấp nguồn RFID Client, rồi máy chủ	Có Wi-Fi `ESP32-Parking`; máy chủ báo `Got IP` và `mDNS: parking.local -> ...`; TFT hiện giao diện; hai thanh chắn đóng
S2	Cấp nguồn Sensor Client, mở `parking.local`	Web hiện sơ đồ 5 ô, `LIVE / SENSOR`
S3	Đặt xe vào ô S2	TFT đổi ô S2 sang đỏ gần như tức thì; web hiện hình xe ở S2 trong ≤ 1 s

#	Thao tác	Hành vi mong đợi
S4	Rê vật nhanh qua cảm biến ($<$ 50 ms)	Trạng thái không đổi
S5	Quẹt thẻ mới ở cổng vào	Thanh chắn vào mở, tự đóng sau 3 s; web có dòng `IN`; nhiệt độ cập nhật
S6	Giữ thẻ trên đầu đọc 1 s	Chỉ một sự kiện được gửi
S7	Quẹt lại thẻ đó ở cổng vào	Không mở; TFT `WRONG GATE – ALREADY INSIDE – USE EXIT GATE` 5 s; web `REJECTED · WRONG GATE`
S8	Quẹt thẻ đó ở cổng ra	Thanh chắn ra mở; TFT hiện hóa đơn 60 s; web `OUT` kèm thời lượng và phí
S9	Đưa lửa gần cảm biến lửa	Còi kêu; TFT `DANGER`; hai thanh chắn mở; băng rôn đỏ trên web
S10	Bấm `CLOSE` trên web khi đang cháy	Bị từ chối, thanh chắn vẫn mở
S11	Tắt lửa	Còi tắt, hai thanh chắn đóng, TFT trở lại bình thường
S12	Bấm `OPEN`/`CLOSE`/`EMERGENCY` trên web	Thanh chắn tương ứng chuyển động, trạng thái trên web đúng
S13	Quẹt thẻ thứ 6 khi đã có 5 xe	Không mở; TFT `PARKING FULL`
S14	Tắt RFID Client rồi bật lại	Máy chủ và Sensor Client tự kết nối lại; `parking.local` trỏ đúng IP mới
S15	Gửi gói UDP rác từ máy tính	Máy chủ ghi cảnh báo và tiếp tục hoạt động

Chương 8. Tổng kết

8.1 Kết quả đạt được

Nhóm đã xây dựng hoàn chỉnh một hệ thống bãi đỗ xe thông minh quy mô mô hình gồm ba vi điều khiển, tự vận hành trên mạng Wi-Fi riêng:

- **10 use case** (Chương 2) đều được hiện thực: theo dõi chỗ trống tại bãi và trên web, gửi/lấy xe bằng RFID kèm tính phí, từ chối quẹt sai cổng hoặc bãi đầy, cảnh báo cháy và mở đường sơ tán, giám sát nước và nhiệt độ, điều khiển thanh chắn thủ công, nhật ký vào/ra kèm giá.
- **Kiến trúc** phân lớp rõ ràng, không vòng phụ thuộc; đa tác vụ với FreeRTOS và ba cơ chế đồng bộ dùng đúng chỗ; giao tiếp UDP nhẹ, tự sửa khi mất gói; tên `parking.local` luôn trỏ đúng IP.
- **Hiệu năng**: màn hình phản ánh thay đổi trong khoảng 0,1 s; thanh chắn mở khoảng 0,2 s sau khi quẹt thẻ; firmware dùng 36% RAM tĩnh, còn 29% flash ứng dụng.
- **Chất lượng**: 250 phép kiểm thử tự động đều đạt, chạy cả mã nguồn hai client trên phần cứng giả lập.

Qua đề tài, nhóm đã vận dụng các kiến thức trọng tâm của môn lập trình nhúng: hệ điều hành thời gian thực, đồng bộ tác vụ, PWM và SPI có DMA, ADC và lọc nhiễu tín hiệu số, lập trình socket, quản lý bộ nhớ hạn chế, xử lý dữ liệu không tin cậy, và kiểm thử phần mềm nhúng không cần phần cứng.

8.2 Hạn chế

1. **Bảo mật**: bản tin UDP và API điều khiển chưa có xác thực; mật khẩu Wi-Fi cố định trong mã nguồn.
2. **Sự kiện RFID có thể mất**: khác bản tin cảm biến, sự kiện quẹt thẻ không được nhắc lại; nếu gói bị mất, người lái phải quẹt lại.
3. **Dữ liệu trong RAM**: phiên gửi xe và nhật ký mất khi máy chủ khởi động lại; thời gian tính từ lúc khởi động, không phải giờ thực.
4. **Nhiệt độ** chỉ được cập nhật khi có xe quẹt thẻ (đi kèm bản tin RFID).
5. **Số phiên RFID và cảm biến IR** chưa được đối chiếu với nhau.

8.3 Hướng phát triển

- Thêm mã xác thực HMAC-SHA256 (có sẵn trong mbedTLS của ESP-IDF) và số thứ tự chống phát lại cho mỗi bản tin; thêm đăng nhập cho API điều khiển.
- Bản tin xác nhận (ACK) cho `rfid_event`; RFID Client gửi lại nếu không nhận được.
- Lưu phiên vào NVS theo lô; đồng bộ giờ bằng SNTP hoặc module RTC DS3231.
- Gửi nhiệt độ định kỳ độc lập với sự kiện quẹt thẻ.
- Kết nối MQTT lên đám mây để quản lý nhiều bãi, thống kê doanh thu, đặt chỗ trước, thanh toán bằng mã QR.
- Bổ sung OTA cho máy chủ (bảng phân vùng hai khe ứng dụng).

Chương 9. Đóng góp của thành viên và phân chia điểm

9.1 Phân công công việc

Mỗi thành viên phụ trách chính một phân hệ, đồng thời cả nhóm cùng tham gia thiết kế kiến trúc, tích hợp, kiểm thử và viết báo cáo. Công việc được chia để khối lượng của ba thành viên tương đương nhau.

Bảng 9.1: Phân công công việc

Thành viên	Công việc chính
Nguyễn Chí Minh 24020232	• Firmware máy chủ: kiến trúc phân lớp, tác vụ FreeRTOS, UDP/HTTP server, mDNS. • Màn hình ST7789, trang web giám sát, nhật ký vào/ra. • Tối ưu hiệu năng và bộ nhớ. • Viết Chương 3, 5 của báo cáo.
Đinh Đức Thịnh 24020313	• Firmware Sensor Client và RFID Client: cảm biến IR, lửa, nước; hai đầu đọc MFRC522, DHT11; OTA. • Logic vào/ra theo cổng, điểm truy cập Wi-Fi, hiệu chỉnh chống rung. • Lắp đặt, đấu dây cảm biến trên mô hình. • Viết Chương 2, 4 của báo cáo.
Nguyễn Việt Tiến 24020322	• Thanh chắn servo tự động, chế độ cháy, còi báo động. • Công cụ giả lập, bộ kiểm thử, kịch bản kiểm thử phần cứng. • Tài liệu hướng dẫn (README), mô hình bãi xe. • Viết Chương 1, 6, 7, 8 của báo cáo.
Cả nhóm	• Phân tích yêu cầu, thiết kế giao thức và kiến trúc tổng thể. • Tích hợp ba bo mạch, chạy kịch bản demo, rà soát báo cáo.

9.2 Tỉ lệ đóng góp và phân chia điểm

Cả ba thành viên thống nhất rằng **mức đóng góp là ngang nhau**. Theo quy định của môn học, toàn bộ thành viên nhận 100% số điểm của báo cáo.

Bảng 9.2: Tỉ lệ đóng góp và tỉ lệ điểm

Thành viên	**MSSV**	**Tỉ lệ đóng góp**	**Tỉ lệ điểm báo cáo**
Nguyễn Chí Minh	24020232	33,3%	100%
Đinh Đức Thịnh	24020313	33,3%	100%
Nguyễn Việt Tiến	24020322	33,3%	100%

Xác nhận của các thành viên:

Nguyễn Chí Minh	Đinh Đức Thịnh	Nguyễn Việt Tiến
_____	_____	_____

Tài liệu tham khảo

[1] Espressif Systems, *ESP32-C3 Series Datasheet* và *ESP32-C3 Technical Reference Manual*.

[2] Espressif Systems, *ESP-IDF Programming Guide* (v6.1): Wi-Fi, LEDC, SPI Master, LCD, NVS, HTTP Server, ESP Timer, FreeRTOS, Partition Tables.

[3] Espressif Systems, *mDNS component* (`espressif/mdns`), ESP Component Registry.

[4] Espressif Systems, *ESP8266EX Datasheet*.

[5] ESP8266 Community, *ESP8266 Arduino Core Documentation*.

[6] Sitronix, *ST7789V — 240RGB×320 dot 262K Color TFT Controller*.

[7] NXP Semiconductors, *MFRC522 Standard Performance MIFARE and NTAG Frontend*, Product Data Sheet.

[8] TowerPro, *SG90 Micro Servo Datasheet*.

[9] Aosong, *DHT11 Humidity & Temperature Sensor Datasheet*.

[10] R. Barry, *Mastering the FreeRTOS Real Time Kernel*, Real Time Engineers Ltd.

[11] S. Cheshire, M. Krochmal, *RFC 6762: Multicast DNS*, IETF, 2013.

[12] J. A. Storer, T. G. Szymanski, "Data compression via textual substitution", *Journal of the ACM*, 29(4), 1982.

[13] Thư viện *MFRC522* (miguelbalboa) và *DHT sensor library* (Adafruit) cho Arduino.